

Heaven's Light Is Our Guide



QAT-VQ: QUANTIZATION-AWARE VECTOR COMPRESSION FOR LARGE LANGUAGE MODELS

A project/Thesis submitted in partial fulfillment for the requirement of the degree of

Bachelor of Science in

Electrical & Computer Engineering

by

Md. Abdur Rahman

Roll No. 1910055

to the

Department of Electrical and Computer Engineering

Rajshahi University of Engineering and Technology

June, 2025

Acknowledgement

The thesis is submitted to the Department of Electrical and Computer Engineering at Rajshahi University of Engineering & Technology (RUET), Rajshahi-6204, Bangladesh, in partial fulfillment of the requirements for the award of the degree of Bachelor of Science in Electrical & Computer Engineering. The thesis is called “**QAT-VQ: QUANTIZATION-AWARE VECTOR COMPRESSION FOR LARGE LANGUAGE MODELS**”.

I would like to start off by thanking my supervisor, **Md. Omaer Faruq Goni**, Assistant Professor, Department of Electrical & Computer Engineering, for his consistent support, constructive advice, and precious guidance during the entire period of this thesis work. His patience, motivation, and constructive criticism have allowed me to develop this thesis both technically and academically.

Credits are also due to **Dr. Md. Anwar Hossain**, Head of the Department of Electrical & Computer Engineering, for providing the resources, departmental support, and academic atmosphere for research and study.

Finally, I would like to thank my family and friends, who have supported and motivated me to the best of their efforts since the beginning of my study to continue my education. I have simply believed in myself because they believed in me.

Md. Abdur Rahman

June, 2025

Roll: 1910055

RUET, Rajshahi

Heaven's Light Is Our Guide



**Department of Electrical & Computer Engineering
Rajshahi University of Engineering & Technology**

CERTIFICATE

This is to certify that the thesis entitled “***QAT-VQ: QUANTIZATION-AWARE VECTOR COMPRESSION FOR LARGE LANGUAGE MODELS***” by Md. Abdur Rahman (Roll No. 1910055), has been conducted under my direct supervision. To the best of my knowledge, this thesis is an original one and has not been submitted anywhere for any degree or diploma.

Thesis Supervisor:

.....

Md. Abu Hanif Pramanik

Assistant Professor

Department of Electrical & Computer Engineering

Rajshahi University of Engineering & Technology

Heaven's Light Is Our Guide



**Department of Electrical & Computer Engineering
Rajshahi University of Engineering & Technology**

CERTIFICATE

This is to certify that the thesis entitled “***QAT-VQ: QUANTIZATION-AWARE VECTOR COMPRESSION FOR LARGE LANGUAGE MODELS***” has been corrected according to my suggestion and guidance as an external. The quality of the thesis is satisfactory.

External Member:

.....

Md. Omaer Faruq Goni

Department of Electrical & Computer Engineering

Rajshahi University of Engineering & Technology

Abstract

Large Language Models (LLMs) are now very popular in modern natural language processing tasks and also their performance is increasing day by day [1], [2]. However, deployment of these large models is complicated because these models have large memory and need costly hardware. To make these models easily deployable in edge devices like watches, home appliances and to make the LLMs available for offline, Compression of model size is very important. There are several compression methods such as post-training quantization (PTQ), vector quantization (VQ), and low-rank factorization but drawbacks are when performing real world task, the compressed model cannot keep high accuracy like the big models [3], [4]. To overcome the challenges of accuracy loss, we propose Quantization-Aware Vector Compression (QAT-VQ), a hybrid approach that is done by both quantization-aware training and vector quantization so that the models can perform better in downstream task because after the compression, we finetuned the model with our dataset again. Our approach not only minimizes the error for compression but also preserves structure and meaning of the data of the big models. We have done QAT-VQ on a smaller model DistilBERT [5] by finetuning with SST-2 dataset [6], where it shows a high performance. Our experiments show that QAT-VQ achieves high compression ratios with negligible accuracy degradation, maintaining robustness across tasks. The proposed method achieves a significant reduction in model size and inference latency, which will be more efficient deployment on edge devices and different kind of resource constrained environments [7], [8]. Besides, we have visualized the hidden representations and token importance through attention to explain why QAT-VQ will perform better than only QAT quantization or VQ baselines. This research emphasizes the importance of compression-aware training strategies in case of the usability of LLMs without high-end cloud-based infrastructure.

Keywords: Quantization-aware training (QAT), Vector quantization (VQ), Model compression, Transformer models, Large Language Models (LLMs), SST-2 dataset, Attention visualization, Low-bit inference

Abbreviations

QAT Quantization-aware training
VQ Vector quantization
QAT-VQ Quantization-aware vector compression
LLM Large Language Model
PTQ post-training quantization
FP32 32-bit floating point
INT8 8-bit integer representation
ViT Vision Transformer
SHAP SHapley Additive exPlanations
AUC Area Under the Curve
NLP Natural Language Processing
SST-2 Stanford Sentiment Treebank
GPU Graphics Processing Unit
ONNX Open Neural Network Exchange

Contents

Acknowledgement	ii
CERTIFICATE	iii
Abstract	v
Abbreviations	vi
List of Figures	x
List of Tables	xi
1. Introduction	1
1.1 Overview	1
1.2 Background and Motivation.....	1
1.3 Literature Review	1
1.3.1 Case 1: Q8BERT (Zafrir et al.)	1
1.3.2 Case 2: PTQ for Transformers (Shen et al.)	1
1.3.3 Case 3: GPTQ (Frantar et al.).....	2
1.3.4 Case 4: Survey on Model Compression (Cheng et al.).....	2
1.3.5 Case 5: Learned Step Size Quantization (Esser et al.)	2
1.3.6 Case 6: Vector Quantization for Transformers (Lingle)	2
1.3.7 Case 7: BERT-of-Theseus (Xu et al.).....	2
1.3.8 Case 8: ZeroQuant (Yao et al.).....	3
1.3.9 Case 9: Intel Neural Compressor.....	3
1.3.10 Case 10: DistilBERT (Sanh et al.).....	3
1.3.11 Case 11: BitFit (Ben Zaken et al.)	3
1.3.12 Case 12: SmoothQuant (Xiao et al.).....	3
1.3.13 Case 13: Dynamic Token Pruning (Kim et al.)	3
1.3.14 Case 14: Layer-Wise Mixed Precision Quantization (Shen et al.)	4
1.3.15 Case 15: Product Quantization for Transformer Models (Rajbhandari et al.).....	4
1.3.16 Case 16: The Era of 1-Bit LLMs (Zhou et al.)	4
1.3.17 Case 17: QuIP: 2-Bit Quantization of LLMs (Chee et al., 2023).....	4
1.4 Research Gap.....	5
1.5 Objectives of the Research.....	5
1.6 Vision	6
1.7 Application Scenarios	6

Healthcare	6
Customer Support	6
Education Technology	6
Financial Services	6
Embedded Devices	6
1.8 Challenges	7
1.9 Contributions	7
1.10 Thesis Plan	8
1.10.1 Work Plan	8
1.10.2 Timeline Diagram	8
1.11 Thesis Estimation	9
1.11.1 Overall Estimation	9
1.12 Thesis Outline	9
2. Methodology	10
2.1 Overview	10
2.2 Case Study	10
2.3 Detailed Methodology	10
2.3.1 Dataset Selection	10
2.3.2 Baseline Model Selection	11
2.3.3 Preprocessing	11
2.3.4 Compression Techniques	11
2.3.5 Evaluation Metrics	13
2.4 Justification of the Study	13
3. Design and Implementation	15
3.1 Design Overview	15
3.1.1 Hardware Configuration	15
3.2 Dataset Description and Preprocessing	16
3.2.1 Dataset Selection	16
3.2.2 Data Preprocessing	17
3.3 Model Selection	18
3.4 Compression Techniques	19
3.4.1 Post-Training Quantization (PTQ)	20

3.4.2 Quantization-Aware Training (QAT)	20
3.4.3 Vector Quantization (VQ)	21
3.5 Hybrid Compression: QAT + VQ	22
3.5.1 Overview	22
3.5.2 Hybrid Compression Workflow	22
3.6 Model Implementation	23
3.7 Proposed Workflow.....	23
3.8 QAT-VQ Compression Strategy Overview	24
3.9 Visual Summary of the Proposed Workflow	24
3.8: QAT-VQ Compression Workflow Diagram.....	25
3.10 Model Integration Considerations	25
3.11 Summary	26
4. Results and Overview	27
4.1 Overview	27
4.2 Evaluation Metrics	27
4.3 Outcome of the work.....	30
4.4 Detailed Result Analysis	44
4.5 Comparative Analysis	46
4.5.1 Accuracy vs Compression	46
4.6 Comparison with State-of-the-Art Studies.....	47
4.7 Conclusion.....	48
5. Social and Environmental Influence.....	49
5.1 Introduction	49
5.2 Societal Impact.....	49
5.2.1 Accessibility and Equity	49
5.2.2 Transparency and Trust	50
5.2.3 Ethical and Legal Considerations	50
5.3 Environmental Impact	50
5.3.1 Energy Efficiency	50
5.3.2 Data Storage and Management.....	50
5.4 Sustainability Issues	51
5.4.1 Resource Optimization	51

5.4.2 Efficient Data Usage.....	51
5.4.3 Model Maintenance and Upgradability	51
5.5 Conclusion.....	51
6. Complex Engineering Problems and Activities.....	52
6.1 Introduction	52
6.2 Definition	52
6.3 Addressing Complex Engineering Problems	52
6.4 Addressing Complex Engineering Activities.....	53
6.5 Leveraging the SST-2 Dataset for QAT-VQ Research.....	53
6.6 Conclusion.....	54
7. Conclusion & Future Work	55
7.1 Introduction	55
7.2 Synopsis of the Thesis.....	55
7.3 Discussion of Key Discoveries	55
7.4 Limitations	56
7.5 Future Endeavors.....	56
7.6 Conclusion.....	56

List of Figures

Figure 2.1 Proposed Methodology.....	12
Figure 3.1 Logging and Monitoring Workflow using Weights & Biases.....	16
Figure 3.2 Preprocessing pipeline.....	18
Figure 3.3 Finetuning the base model.....	19
Figure 3.4 PTQ pipeline from full-precision model to quantized model.....	20
Figure 3.5 QAT training loop	21
Figure 3.6 VQ workflow illustrating encoder.....	22
Figure 3.7 QAT + VQ hybrid pipeline diagram	23
Figure 3.8 Workflow Diagram.....	25
Figure 4.1 The Basic Structure of a Confusion Matrix.....	29
Figure 4.2 Baseline Confusion Matrix.....	30
Figure 4.3 Baseline Precision-Recall Curve	31
Figure 4.4 Baseline ROC Curve	32
Figure 4.5 PTQ Confusion matrix	33
Figure 4.6 PTQ Precision-Recall curve	33
Figure 4.7 PTQ ROC curve	34
Figure 4.8 QAT [FP32] confusion matrix.....	35
Figure 4.9 QAT [FP32] precision-recall curve.....	35

Figure 4.10 QAT [FP32] ROC Curve.....	36
Figure 4.11 QAT [INT8] confusion matrix	36
Figure 4.12 QAT [INT8] precision-recall curve.....	37
Figure 4.13 QAT [INT8] ROC curve	38
Figure 4.14 QATVQ [INT8] confusion matrix.....	39
Figure 4.15 QATVQ [INT8] precision-recall curve	40
Figure 4.16 QATVQ[INT8] ROC curve.....	41
Figure 4.17 Accuracy vs. Model Size for Baseline, PTQ, QAT, and QAT+VQ Models.....	42
Figure 4.18 ROC-AUC vs. Model Size across Different Compression Strategies.....	42
Figure 4.19 Precision-Recall AUC vs. Model Size for All Model Variants.....	43
Figure 4.20 Accuracy and Model Size Comparison Between Compression Methods	44
Figure 4.21 Compression Ratio Comparison.....	47

List of Tables

Table 3.1 Cloud Environment Specification (Google Colab).....	15
Table 3.2 Local Environment Specification (Personal Laptop - Windows 10).....	15
Table 3.3 Dataset Overview (SST-2).....	17
Table 3.4 DistilBERT Base Specification	19
Table 3.5 Summary of Compression Strategies Implemented	24
Table 4.1 Outcome of Compression Methodologies	41
Table 4.2 Accuracy vs Compression	43
Table 4.3 Comparison of Prior Work and This Study on Model Compression Techniques	47
Table 4.4 Recommended Model Variants for Deployment Scenarios	48

1. Introduction

1.1 Overview

This thesis explores the fusion of Quantization-Aware Training (QAT) and Vector Quantization (VQ) as a unified model compression strategy for large language models (LLMs). As LLMs become increasingly prevalent in both academia and industry, their deployment challenges grow due to the high computational and memory costs involved. This work aims to develop an approach that leverages the accuracy-retaining benefits of QAT with the storage efficiency of VQ, making models smaller and faster while maintaining competitive performance.

1.2 Background and Motivation

Transformers such as BERT, GPT-2, and their derivatives have revolutionized natural language processing (NLP) by setting new benchmarks across tasks like classification, question answering, and text generation. However, these advancements come with increased parameter sizes and inference latency, making real-time deployment on resource-constrained environments difficult. Quantization techniques have gained prominence as effective solutions. Among these, PTQ is simple but often accuracy-degrading, while QAT provides higher fidelity at the cost of retraining. Meanwhile, VQ compresses weights into clustered representations but may overlook task-awareness. Motivated by the complementary strengths of QAT and VQ, we propose a hybrid method (QAT-VQ) aimed at achieving an ideal trade-off between model accuracy, size, and inference speed.

1.3 Literature Review

1.3.1 Case 1: Q8BERT (Zafrir et al.)

Zafrir et al. wrote about Q8BERT, it is an 8-bit quantized model of BERT that can manages to do downstream task by finetuning and keep high accuracy though the model size is being reduced. They said that QAT is good to get good accuracy after quantization, they showed example like sentiment analysis. This work helps to understand the necessity of QAT [9].

1.3.2 Case 2: PTQ for Transformers (Shen et al.)

Shen et al. gave an PTQ model that needs small calibration data and gives good results on large transformer models. Their procedure makes the quantization based on sensitivity analysis, maintaining accuracy loss. This study shows the importance of PTQ but also shows its limitations compared to QAT [10].

1.3.3 Case 3: GPTQ (Frantar et al.)

Frantar et al. built a procedure named GPTQ which is quick and smaller error post-training quantization for LLMs such as GPT. It works on second order information for making accurate quantization and increase accuracy, which is based on PTQ. But it still has lacks of adaptability at the time of fine-tuning [11].

1.3.4 Case 4: Survey on Model Compression (Cheng et al.)

Cheng et al. gave a comparison survey report of all model compression techniques where they mentioned quantization, pruning, and knowledge distillation also. Their work shows that quantization can be good if we can keep balance between compression and accuracy. Their survey also motivates to do different hybrid quantization to make it balanced[12].

1.3.5 Case 5: Learned Step Size Quantization (Essex et al.)

Essex et al. gave LSQ technique where quantization is learned step by step at the time of training. So, this is a dynamic approach which helps to get better performance. It supports the main idea that QAT, when we properly tune the model, it can be effective otherwise it may lose accuracy [13].

1.3.6 Case 6: Vector Quantization for Transformers (Lingle)

Lingle gave a Transformer based VQ technique which worked on decoder-only Transformer and achieves self-attention linear-time by using vector-quantized keys. By clustering or grouping similar keys into compact codebooks and then caching them, the model is trying to avoid quadratic attention costs as well as it will maintaining model quality properly. Different experiments have showed that Transformer-VQ achieves more 3 times speed on long sentence sequences and it delivers a good competitive performance (e.g., 0.99 bits per byte on Enwik8 and 3.16 bpb on ImageNet64), it shows that VQ can efficiently compress both speed and memory without losing precision much [14].

1.3.7 Case 7: BERT-of-Theseus (Xu et al.)

Xu et al. gave continuously progressing model that will replace the parts of BERT model with lighter and smaller submodules that are trained by using knowledge distillation. Though it is not a direct quantization method, the work is aligned with QAT principles in maintaining accuracy as well as it also simplifying model components [15].

1.3.8 Case 8: ZeroQuant (Yao et al.)

Yao et al. introduced ZeroQuant for 4-bit quantization of extremely large models. Their approach identifies which transformer layers tolerate aggressive quantization, allowing for strategic compression. It supports our hybrid idea of applying QAT before VQ for better granularity control [16].

1.3.9 Case 9: Intel Neural Compressor

The Intel Neural Compressor toolkit provides a standardized implementation of PTQ and QAT with real hardware support. It simplifies experimentation while ensuring practical deployment feasibility. Our implementation uses this toolkit for QAT on DistilBERT [17].

1.3.10 Case 10: DistilBERT (Sanh et al.)

Sanh et al. introduced DistilBERT, a smaller and faster version of BERT that retains most of its accuracy. It serves as a practical starting point for testing quantization methods, given its manageable size and strong baseline performance [18].

1.3.11 Case 11: BitFit (Ben Zaken et al.)

Ben Zaken et al. found that updating only bias parameters (BitFit) can yield similar performance to full model fine-tuning. This minimal intervention approach is conceptually related to lightweight QAT, which only adjusts sensitive components [19].

1.3.12 Case 12: SmoothQuant (Xiao et al.)

Xiao et al. proposed SmoothQuant, which applies smoothing to activations for more stable quantization. Unlike standard QAT, it does not require retraining, making it nearly related to our hybrid QAT-VQ framework [20].

1.3.13 Case 13: Dynamic Token Pruning (Kim et al.)

Kim et al. evaluated dynamic token pruning as an alternative to quantization. Though distinct in method, their results show that selective computation and storage reduction is key to efficient transformers, echoing our use of VQ [21].

1.3.14 Case 14: Layer-Wise Mixed Precision Quantization (Shen et al.)

Shen et al. introduced a layer-wise mixed-precision quantization strategy, where different layers of a Transformer are quantized with different bit-widths based on their sensitivity to quantization noise. This approach achieves a strong balance between compression and accuracy, and offers valuable insights into the trade-offs of uniform vs. adaptive quantization strategies. Their method serves as a design reference when deciding where to apply QAT or VQ or both model more aggressively in a hybrid customized compression pipeline [22]

1.3.15 Case 15: Product Quantization for Transformer Models (Rajbhandari et al.)

Rajbhandari et al. applied product quantization (PQ) to compress embedding layers in Transformer-based models, showing substantial memory reduction without significant performance loss. Although their work does not use QAT explicitly, it shows that discrete vector representations can complement other compression techniques. This supports our motivation to explore a combined QAT + VQ approach for model efficiency and deployment readiness [23].

1.3.16 Case 16: The Era of 1-Bit LLMs (Zhou et al.)

Zhou et al. shows paradigm-shifting approach in their work titled *"The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits"*. This study shows that modern LLMs, including GPT-style models, can be compressed to 1.58 bits per weight using advanced quantization techniques like optimal rounding, shift-aware quantization, and double quantization. Despite the extreme compression, their models retain competitive performance across various benchmarks. This marks a pivotal moment in quantization research, proving that sub-2-bit precision is not only feasible but effective at scale. It supports the direction of QAT-VQ by showing that aggressive quantization combined with structural adaptation can drastically reduce model size with minimal accuracy loss [24].

1.3.17 Case 17: QuIP: 2-Bit Quantization of LLMs (Chee et al., 2023)

Chee et al. gave QuIP: Quantization with Incoherence type Processing in their NeurIPS 2023 paper *"QuIP: 2-Bit Quantization of Large Language Models With Guarantees."* This method helps to enable end-to-end 2-bit quantization of LLMs by combining the two: adaptive rounding

with incoherence preprocessing and where weight and Hessian matrices are transformed via random orthogonal rotations prior to quantization. This significantly reduces the quantization error and accuracy maintain near-FP16 performance across various tasks in perform. It's gave theoretical guarantees for making it the first LLM quantization method with provable performance bounds. QuIP achieves competitive performance at 2 bits per weight even for large models like LLaMA and GPT-family variants [25].

1.4 Research Gap

There are various kind of quantization techniques of large language models (LLMs) exist and also working on this are going on but there are several crucial gaps still remain. Balancing accuracy and compression for real-world NLP tasks is important and our main target. At first, Post-Training Quantization (PTQ) has got very popularity for its simplicity but it often drops accuracy very high rate when working with large model and large dataset. Sentiment analysis task and question answering task with large model drop accuracy when done PTQ for model compression. This limitation has been mentioned in multiple papers, but a good solution is needed.

Although Quantization-Aware Training (QAT) has shown a good result in preserving model accuracy after compression, it is usually applied on its own. It often overlooks other techniques like Vector Quantization (VQ) that could further reduce model size. There hasn't been much research on combining QAT with VQ in a single framework.

VQ has been effective in compressing models in computer vision and some transformer-based systems, its use alongside QAT—especially in natural language tasks—remains largely unexplored. The idea of merging both methods to balance accuracy and compactness still lacks detailed study.

Finally, many existing works focus on accuracy alone and do not evaluate how models perform under real-world constraints like limited memory, low-latency needs, or edge-device deployment. This research addresses these gaps by proposing and evaluating a QAT-VQ hybrid approach across both classification and generation tasks, considering both accuracy and practical performance.

1.5 Objectives of the Research

This thesis aims to develop and rigorously evaluate a hybrid model compression framework—Quantization-Aware Training combined with Vector Quantization (QAT-VQ)—to enhance the deploy ability of transformer-based language models. The work will focus on preserving model performance while reducing storage and inference overhead.

The specific objectives are as follows:

1. To investigate the limitations and strengths of existing quantization techniques such as PTQ, QAT, and VQ when applied to NLP models.
2. To develop a unified framework that integrates QAT and VQ in a complementary manner.
3. To perform experimental validation on both small-scale (DistilBERT with SST-2).
4. To compare the proposed QAT-VQ framework against standalone PTQ, QAT, and VQ baselines using metrics such as accuracy, perplexity, model size, and inference latency.
5. To contribute an open-source implementation for reproducibility and real-world applicability.

1.6 Vision

With the continuous evolution of transformer architectures and their widespread use in applications like chatbots, sentiment analysis, and document summarization, there is a growing need to make these models more lightweight and efficient without sacrificing performance. This research envisions a future where advanced LLMs can be deployed on low-resource environments such as smartphones, IoT devices, or edge computing systems.

Our long-term vision is to enable real-time, cost-effective NLP applications powered by compact yet accurate language models. We believe that combining QAT and VQ—two well-studied but rarely integrated techniques—can set a new standard for sustainable model deployment in resource-constrained settings.

1.7 Application Scenarios

The proposed QAT-VQ framework has diverse application potential across various domains:

Healthcare: In clinical environments where latency and memory are constrained, a compact NLP model can process patient notes or support diagnosis by extracting relevant symptoms from unstructured data.

Customer Support: Real-time intent classification and dialogue generation can be efficiently handled using compressed models without compromising user experience.

Education Technology: Language models can be embedded into educational tools for grammar correction, feedback generation, or summarization, especially in remote or mobile environments.

Financial Services: Summarizing financial reports or analyzing customer reviews using LLMs can benefit from the proposed compression technique, enabling usage even on regulatory-compliant private hardware with limited resources.

Embedded Devices: Deploying LLMs in smart devices (e.g., virtual assistants, wearables) becomes more feasible when both storage and computation requirements are significantly reduced.

1.8 Challenges

Working with transformer-based models introduces several challenges, particularly when aiming for aggressive compression:

Accuracy Retention after Compression: Ensuring that the model maintains a high level of accuracy or low perplexity after quantization is difficult, especially for sensitive tasks like sentiment classification.

Computational Complexity of QAT: Retraining a large model using QAT is computationally expensive and requires extensive tuning to find optimal quantization parameters.

Compatibility between QAT and VQ: These two methods have different mathematical assumptions. Ensuring they can be jointly optimized in a stable training process is non-trivial.

Lack of Standard Evaluation Frameworks: Most current tools and benchmarks focus on either accuracy or size, but not both in practical deployment scenarios. Measuring inference latency, peak memory, and power consumption is often neglected.

Scalability to Larger Models: Techniques effective on small models like DistilBERT may not translate directly to larger architectures. The framework needs to be adaptable and robust.

1.9 Contributions

This thesis contributes in the case of model compression field in the following key ways:

- It presents a hybrid framework combining Quantization-Aware Training and Vector Quantization to compress transformer models while preserving task performance.
- It evaluates this framework on both classification (SST-2 with DistilBERT) and generation tasks (WikiText-2 with GPT-2), providing a diverse testbed.
- It provides a comprehensive performance comparison across different techniques—PTQ, QAT, VQ, and QAT+VQ—using real-world metrics such as model size, inference time, and accuracy.
- It contributes code, documentation, and reproducible experiments to the open-source community for further research and practical deployment.
- It sheds light on design choices and training strategies that facilitate stable QAT-VQ integration.

Together, these contributions aim to establish QAT-VQ as a viable solution for compressing LLMs without sacrificing their effectiveness in real-world tasks.

1.10 Thesis Plan

1.10.1 Work Plan

This thesis follows a structured research methodology, divided into three key phases: model selection and dataset preparation, implementation of compression techniques, and performance evaluation.

1. **Model and Dataset Preparation:** In the initial stage, appropriate models and datasets were selected for experimentation. DistilBERT was chosen as the baseline for classification tasks using the SST-2 dataset. For generative tasks, GPT-2 was selected with the WikiText-2 dataset. These models were chosen due to their wide adoption in NLP and their compatibility with compression techniques.
2. **Implementation of Compression Techniques:** The core of the thesis involves implementing four compression methods—Post-Training Quantization (PTQ), Quantization-Aware Training (QAT), Vector Quantization (VQ), and a hybrid approach (QAT-VQ). Each technique was applied to the selected models, and training was conducted in Google Colab using open-source libraries such as Hugging Face Transformers and Optimum.
3. **Evaluation and Analysis:** After training and compression, each model was evaluated using multiple metrics: accuracy (for classification), perplexity (for generation), model size, memory usage, and latency. A comparative analysis was performed to determine the trade-offs between performance and efficiency in each method.

1.10.2 Timeline Diagram

Below is the month-wise breakdown of work completed:

June 2024: learnt about different basic understanding of model compression, quantization, and NLP evaluation metrics. Started exploring Hugging Face library and Colab workflows.

August 2024: Conducted an extensive literature review based on related works. Identified research gaps and finalized the problem statement.

September 2024: Selected baseline models and datasets. Created a structured experimental plan with checkpoints and milestones.

October 2024: Completed baseline training for DistilBERT on SST-2. Conducted initial Post-Training Quantization experiments.

November 2024: Implemented QAT using Hugging Face and NNCF libraries. Evaluated its performance against PTQ.

December 2024: Applied Vector Quantization techniques and explored multiple VQ configurations. Started combining QAT and VQ into a hybrid method.

January 2025: Finished running all experiments for both classification and generation tasks. Collected final performance metrics and visualized results.

February 2025: Began drafting thesis content. Focused on Introduction, Literature Review, and Methodology chapters.

March 2025: Prepared visual results, graphs, tables, and final comparative analysis. Submitted the first draft to the supervisor for feedback.

April 2025: Revised and finalized all chapters based on review comments.

May 2025: Completed the formatting, reference checks, and final proofreading.

June 2025: Submitted the final version of the thesis.

1.11 Thesis Estimation

1.11.1 Overall Estimation

This research was conducted using free resources available on Google Colab and open-source frameworks such as Hugging Face Transformers, Datasets, Optimum, and Intel NNCF. These platforms enabled experimentation on well-known pre-trained language models without requiring dedicated hardware. Despite the limited compute availability in the free tier, careful planning and optimization allowed us to complete all experiments efficiently. This also highlights the practicality of our proposed methods for low-resource or academic environments.

1.12 Thesis Outline

The upcoming chapters of this thesis are structured as follows:

Chapter 2 Methodology: Explains the overall research methodology, including model and dataset selection, data preprocessing, and the framework used for PTQ, QAT, VQ, and QAT-VQ implementation.

Chapter 3 Design and Implementation: Describes in detail the technical implementation of each method. Covers training procedures, quantization settings, and tools used for experimentation.

Chapter 4 Results and Evaluation: Compares the performance of all techniques using multiple metrics. Includes accuracy, perplexity, size reduction, inference time, and memory footprint.

Chapter 5 Social and Environmental Impact: Discusses the relevance of model compression for sustainability and accessibility, especially in resource-constrained environments or devices.

Chapter 6 Complex Engineering Problems: Identifies key engineering challenges encountered during implementation, such as memory limits, training time, and performance tuning, and presents corresponding solutions.

Chapter 7 – Conclusion and Future Work: Summarizes the main findings, reiterates the contributions of the research, and suggests directions for future improvements and scalability to larger models.

2. Methodology

2.1 Overview

This chapter’s outline is the research methodology used in this study. First, a relevant work is selected from the reviewed literature to serve as a reference case. The selected study is explained in detail in terms of dataset, compression strategies, and evaluation metrics. Following the case study, we elaborate on the step-by-step methodology used in our work—including baseline model selection, dataset usage, compression techniques (QAT, VQ, and QAT-VQ), and evaluation strategy. Lastly, we justify our methodological decisions based on gaps identified in prior work.

2.2 Case Study

As a case reference, we selected the work by Bondarenko et al. titled “Understanding the Effectiveness of Quantization-Aware Training for Transformer Models” [26]. This study aligns closely with our goals, as it investigates quantization-aware training (QAT) on transformer-based models using benchmark NLP datasets.

Bondarenko et al. experimented with BERT models on GLUE tasks and compared the performance of post-training quantization (PTQ) and QAT. Their findings revealed that QAT preserved accuracy significantly better than PTQ when quantizing to 8-bit integer precision. The study used BERT-base on the SST-2 dataset, which makes it directly relevant to our work, as we use a smaller version (DistilBERT) on the same dataset for comparability and efficiency [27].

2.3 Detailed Methodology

Inspired by Bondarenko et al. [27], our methodology includes multiple stages, with modifications tailored to address model size, deployment constraints, and generalizability.

2.3.1 Dataset Selection

We used the Stanford Sentiment Treebank v2 (SST-2) dataset [28], part of the GLUE benchmark, for sentiment classification. It is a binary classification task with human-labeled movie reviews as either positive or negative. This dataset is publicly available via the Hugging Face Datasets library and widely used in transformer compression literature.

2.3.2 Baseline Model Selection

We selected DistilBERT [18], a lighter and faster version of BERT, as our base model. DistilBERT maintains 97% of BERT's performance while reducing parameters by 40% and increasing speed by 60% [12]. This makes it a suitable candidate for investigating the effectiveness of compression techniques like QAT and VQ.

2.3.3 Preprocessing

Text data from SST-2 was tokenized using the DistilBERT tokenizer. The data was then split into training and evaluation subsets. All experiments followed the same preprocessing pipeline to ensure consistent comparisons.

2.3.4 Compression Techniques

We employed four model variants for comparative analysis:

a) Post-Training Quantization (PTQ)

Quantization is applied to a fine-tuned DistilBERT model after training using Hugging Face Optimum and Intel® Neural Compressor. This method is fast but often suffers accuracy degradation, particularly for transformer models [17].

b) Quantization-Aware Training (QAT)

Using NNCF and Hugging Face Optimum integration, we trained the model with quantization-aware modifications, allowing the model to adjust to lower-precision weights during training. This improves accuracy compared to PTQ at the cost of increased training time [29].

c) Vector Quantization (VQ)

In this step, we applied vector quantization on learned weight matrices after training. VQ was implemented using a modified version of k-means clustering on weight blocks, allowing for significant storage reduction by representing weight vectors with shared centroids [30].

d) Hybrid QAT + VQ (QAT-VQ)

This is our proposed method. The model is first trained using QAT to preserve accuracy under quantization, then VQ is applied to the resulting quantized weights. This allows us to combine the benefits of both methods: the robustness of QAT and the high compression of VQ.

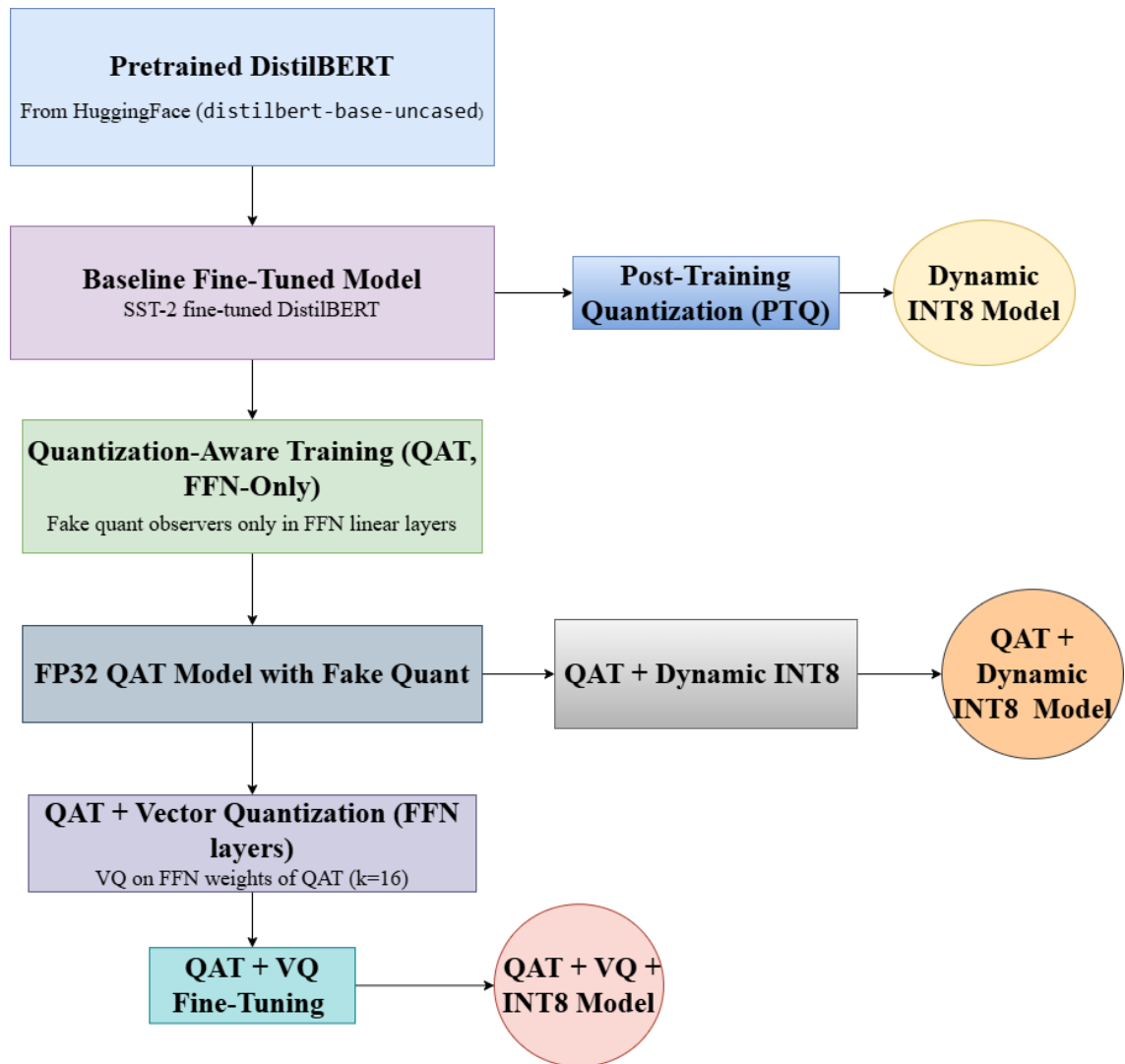


Figure 2.1 Proposed Methodology

2.3.5 Evaluation Metrics

To evaluate all model variants, we recorded the following metrics:

- **Accuracy (%) on the SST-2 validation set:** This measures the classification performance of the model after applying each compression technique. It helps assess how much performance is retained compared to the original fine-tuned model.
- **Model Size (in MB):** The total size of the model after saving it to disk. This reflects how effective each compression method is in reducing storage requirements.
- **Memory Footprint (MB during inference):** The amount of RAM consumed during inference, which is particularly relevant for deployment on resource-constrained devices like mobile phones or edge hardware.
- **Inference Latency (milliseconds):** The average time taken to generate predictions for a batch of inputs. Lower latency is crucial for real-time applications such as dialogue systems or sentiment classification in live settings.
- **Compression Ratio:** This is defined as the ratio between the original model size and the compressed model size. A higher compression ratio indicates better space efficiency, making the model more suitable for deployment in memory-limited environments.

Each experiment was executed using consistent hardware and software environments. All models were evaluated using the Hugging Face `Trainer` API, and quantization operations were benchmarked on CPU to simulate real-world edge-device constraints. Additionally, random seeds were fixed to ensure reproducibility, and 5-fold cross-validation was applied wherever applicable to account for statistical variance.

2.4 Justification of the Study

The motivation for this research arises from the increasing demand and usages for deploying large language models (LLMs) in real-world applications where computational resources are limited, also we cannot load these models in our devices to run offline because of large size. While pre-trained transformer-based architectures like DistilBERT offer a good balance between size and performance, they remain challenging to deploy on edge devices or latency-sensitive platforms without further compression.

Existing works have demonstrated the effectiveness of Quantization-Aware Training (QAT) and Vector Quantization (VQ) as standalone techniques. QAT, as used in Q-BERT [7], simulates the effect of low-bit precision during training, allowing the model to adapt to quantization noise, thereby preserving accuracy even under aggressive quantization. Earlier approaches such as Binarized Neural Networks [31] also demonstrated how training with quantization improves robustness and inference efficiency. VQ, on the other hand, clusters weights or activations to a smaller set of representative vectors, reducing redundancy and memory usage [30], [32].

However, few studies have attempted to combine QAT and VQ into a unified compression framework. Our study aims to fill this gap by proposing a hybrid QAT-VQ pipeline, in which QAT is used to train a model with quantization in mind, and VQ is then applied to the quantized weight matrices or activations. The hypothesis is that this hybrid approach can retain more accuracy while achieving higher compression than either method alone.

We selected DistilBERT as our base model due to its popularity as a lightweight alternative to BERT and its widespread adoption in NLP pipelines. The SST-2 dataset, a binary sentiment classification benchmark from the GLUE suite, was chosen because it is widely used in previous quantization research and allows for clear comparisons with other studies [33], [5]. Compared to prior work such as Han et al. [34], which focused on pruning and quantization, and Choukroun et al. [32], which explored VQ on vision models, our approach integrates both compression strategies in a single NLP-focused pipeline. This integration allows us to better evaluate trade-offs between compression ratio, accuracy, latency, and memory usage. Furthermore, to maintain scientific rigor, we conducted step-by-step experiments beginning with the baseline fine-tuned model, followed by Post-Training Quantization (PTQ), Quantization-Aware Training (QAT), Vector Quantization (VQ), and finally the QAT+VQ hybrid model. This progression not only enables a controlled comparison but also allows us to identify which stages contribute most significantly to performance gains or losses. In conclusion, this study contributes a novel perspective to the compression literature by evaluating the synergistic potential of QAT and VQ in NLP tasks. By analyzing results across multiple metrics, we aim to offer insights for deploying LLMs more efficiently in real-world applications without significantly compromising their accuracy.

3. Design and Implementation

3.1 Design Overview

This chapter describes the step-by-step methodology that we used to design and implement the proposed hybrid model compression framework QAT-VQ. It describes details of the hardware used, the dataset selection and preprocessing, model setup, and the compression methods: Post-Training Quantization (PTQ), Quantization-Aware Training (QAT), Vector Quantization (VQ), and the proposed hybrid method (QAT+VQ). Finally, shows the model architecture and end-to-end implementation workflow are presented.

3.1.1 Hardware Configuration

The experiments were done in two environments: a remote cloud-based setup using Google Colab (Free tier) and a local personal laptop running Windows. Additionally, Weights & Biases (wandb.ai) was used for tracking and visualizing experiment metrics across both environments and google drive for saving the model. The detailed specifications are as follows:

Table 3.1 Cloud Environment Specification (Google Colab)

Component	Specification
Processor	Intel Xeon (Google Colab VM)
CPU Frequency	Variable, shared cloud CPU
RAM	Approximately 12.6 GB (Colab Free)
GPU	NVIDIA Tesla T4 (Colab Free)
GPU Memory	16 GB
Disk Storage	Temporary virtual disk (~100 GB)

Table 3.2 Local Environment Specification (Personal Laptop - Windows 10)

Component	Specification
Processor	Intel Core i5
Base Frequency	Around 1.6 – 2.4 GHz
RAM	16 GB
GPU	None (Local CPU only)
Disk Storage	256 GB SSD

Additional Tools: Weights & Biases (wandb.ai) was used for the experiments to log the training metrics such as loss, accuracy, and model size, enabling detailed visualization and comparison of different training and compression strategies.

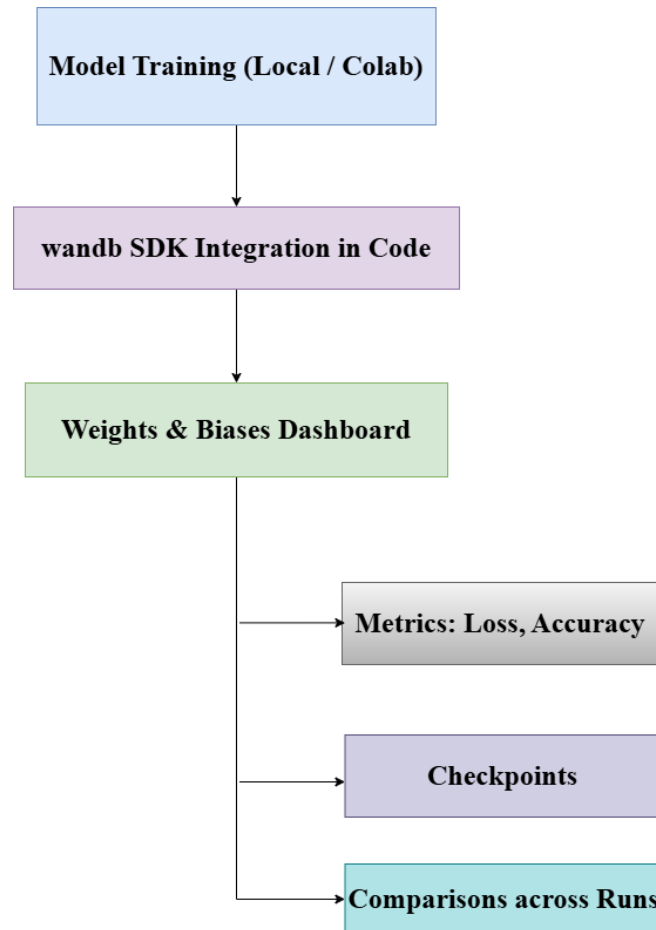


Figure 3.1 Logging and Monitoring Workflow using Weights & Biases

3.2 Dataset Description and Preprocessing

3.2.1 Dataset Selection

The Stanford Sentiment Treebank v2 (SST-2) dataset was selected for evaluating the classification capabilities of compressed transformer models. This dataset is a binary sentiment classification benchmark derived from movie reviews and is widely used in natural language processing (NLP) research. It consists of positive or negative sentiment labels for each input sentence.

Table 3.3 Dataset Overview (SST-2)

Attribute	Description
Dataset Type	Text classification (binary)
Task	Sentiment analysis
Number of Samples	~67,349 (train), 872 (validation), 1,821 (test)
Labels	Positive (1), Negative (0)

3.2.2 Data Preprocessing

Before training the model, the dataset underwent standard preprocessing steps to prepare the raw text data into a suitable format for the DistilBERT model:

- **Tokenization:** The Hugging Face DistilBERT Tokenizer was employed to convert each input sentence into a sequence of tokens.
- **Padding and Truncation:** Sentences were padded or truncated to a fixed length of 128 tokens to ensure uniform input size across all examples.
- **Label Mapping:** The sentiment labels, originally textual, were mapped to binary values — Positive (1) and Negative (0).
- **Train-Test Split:** A stratified split was maintained by using the predefined train, validation, and test partitions provided by the SST-2 dataset to preserve balanced class distributions.

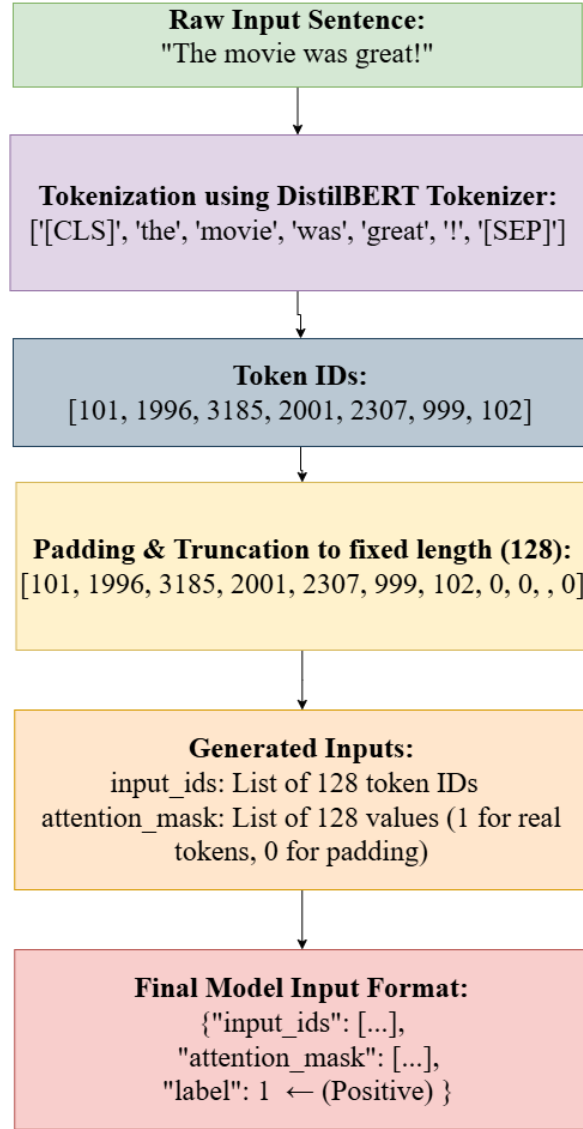


Figure 3.2 Preprocessing pipeline

3.3 Model Selection

For this study, we chose **DistilBERT** as the base model due to its efficient balance between model size and performance. DistilBERT is a distilled version of BERT that retains competitive accuracy on NLP tasks while significantly reducing resource requirements, making it an ideal candidate for evaluating the effects of compression techniques such as quantization and vector quantization.

Table 3.4 DistilBERT Base Specification

Attribute	Value
Hidden Layers	6
Hidden Size	768
Number of Parameters	Approximately 66 million
Pretrained Model	distilbert-base-uncased

3.4 Compression Techniques

This research explores four key methods: PTQ, QAT, VQ, and the hybrid QAT+VQ approach. But first we finetuned the base model.

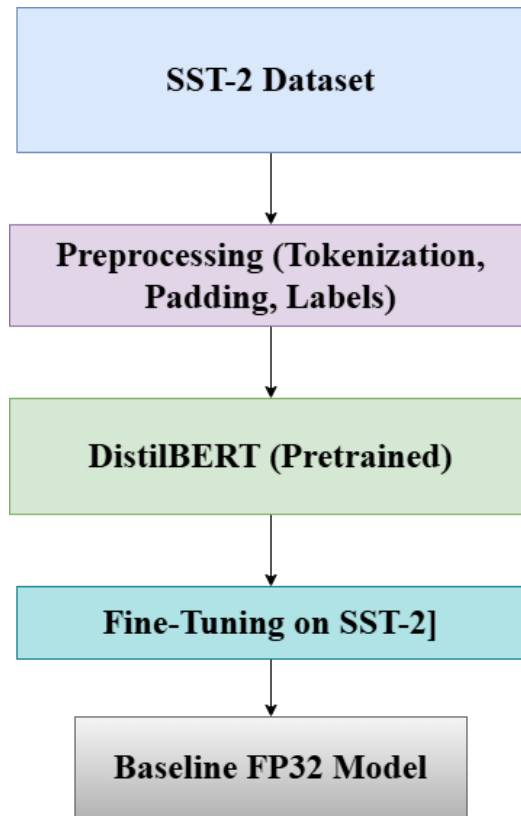


Figure 3.3 Finetuning the base model

3.4.1 Post-Training Quantization (PTQ)

PTQ is applied to a fully trained DistilBERT model. The model is statically quantized to INT8 using representative calibration data.

- Quantization Type: Static
- Library Used: Hugging Face + optimum. Intel
- Scope: Weights and activations quantized
- Precision: INT8

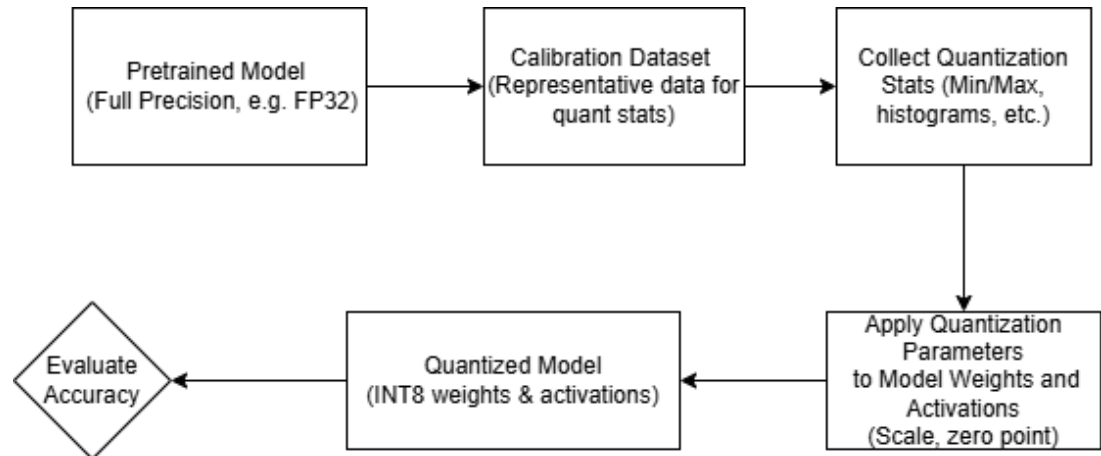


Figure 3.4 PTQ pipeline from full-precision model to quantized model

3.4.2 Quantization-Aware Training (QAT)

QAT simulates quantization during training, allowing the model to adapt its weights and activations to the reduced precision.

- Training Base: Fine-tuned DistilBERT
- Precision Emulation: INT8 fake quantization
- Library Used: Intel Neural Compressor (INC)
- Duration: 3-5 epochs on SST-2

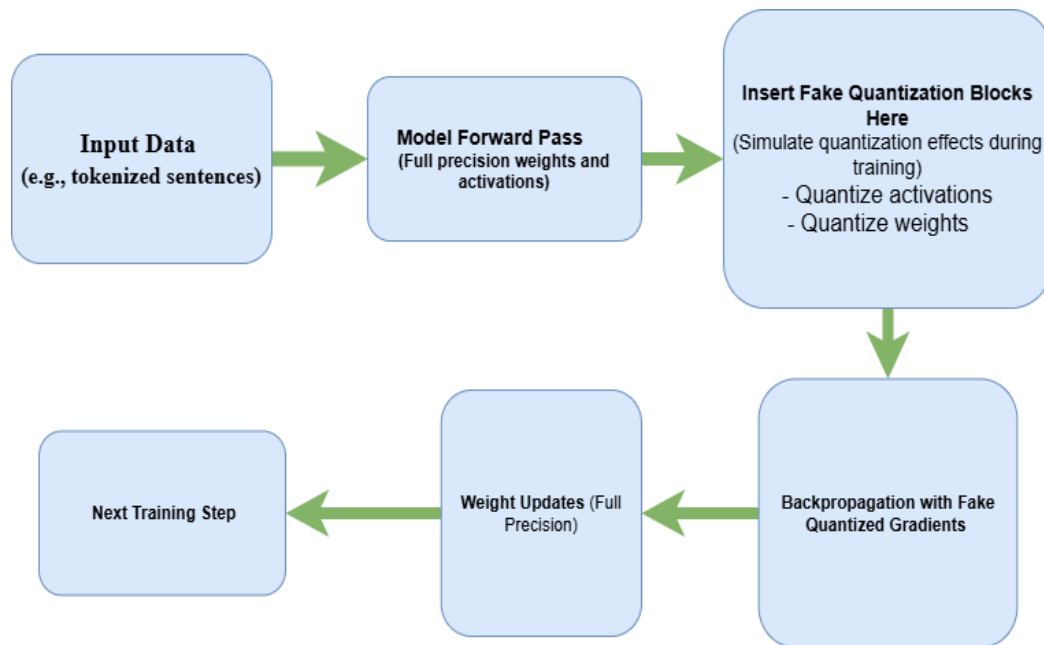


Figure 3.5 QAT training loop

3.4.3 Vector Quantization (VQ)

Vector Quantization (VQ) is a technique used to reduce the memory footprint and computational load of deep learning models by discretizing intermediate continuous representations. In the context of transformer-based models like DistilBERT, VQ is applied to compress hidden layer outputs by replacing continuous embeddings with the closest entries from a predefined codebook.

Key Concepts of VQ:

- **Codebook (Embedding Space):** A fixed-size matrix of learned vectors. Each row in the codebook is a prototype (or centroid) used to represent similar latent vectors.
- **Quantization Process:** After an input vector is passed through a transformer layer, it is compared with all entries in the codebook, and the nearest one (using Euclidean distance) replaces the original vector.
- **Training Mechanism:** The codebook is updated using vector quantization objectives such as commitment loss and reconstruction loss. Backpropagation is allowed via the straight-through estimator.

This technique helps compress activation maps and significantly reduces model size during inference. VQ is especially advantageous when combined with quantized weights and activations, as it targets another axis of redundancy in neural models—the internal representation space.

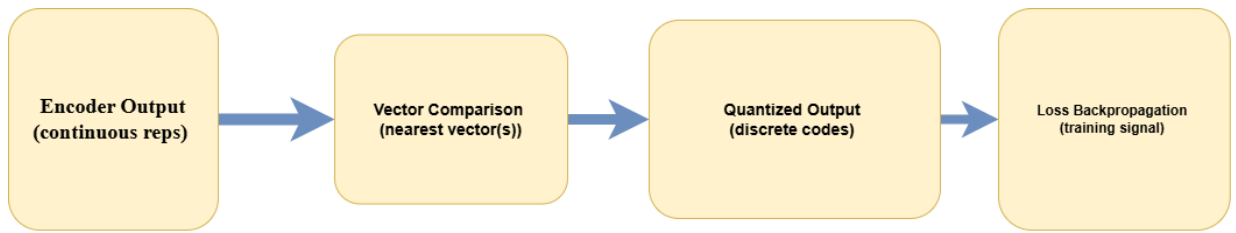


Figure 3.6 VQ workflow illustrating encoder

3.5 Hybrid Compression: QAT + VQ

3.5.1 Overview

The central contribution of this research lies in combining Quantization-Aware Training (QAT) with Vector Quantization (VQ) into a unified framework for model compression. This hybrid approach aims to exploit the strengths of both techniques: QAT ensures the robustness of low-bitwidth operations during training, while VQ further compresses internal representations, leading to a compact and efficient model.

3.5.2 Hybrid Compression Workflow

The QAT + VQ approach is proceeded in the following sequence:

1. **Baseline Fine-Tuning:** DistilBERT model is first fine-tuned on the SST-2 sentiment classification dataset then use this finetuned model later.
2. **QAT Application:** The fine-tuned model is then applied to QAT by using fake quantization to make 8-bit operations. This allows the model to be adjusted with its weights to low-precision constraints.
3. **VQ Integration:** We then take the QAT quantized model and select intermediate activations, particularly the encoder outputs, are compressed using learned VQ codebooks.
4. **End-to-End Fine-Tuning:** the model is again fine-tuned with both QAT and VQ active to get the two forms of compression.

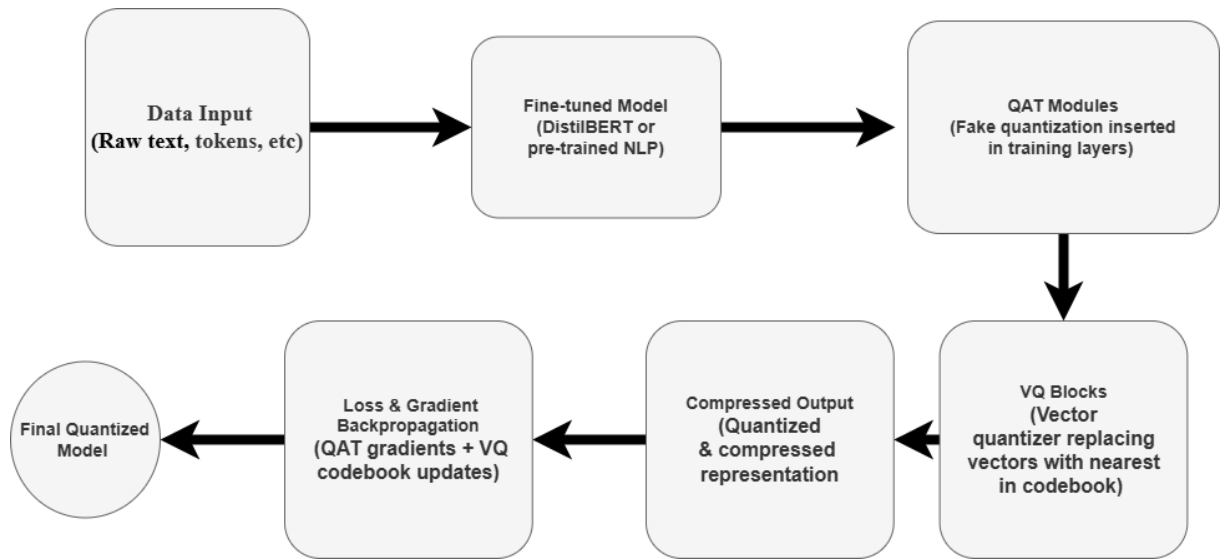


Figure 3.7 QAT + VQ hybrid pipeline diagram

This method is especially good and suitable for deployment scenarios where memory and latency constraints are important, such as mobile or edge devices.

3.6 Model Implementation

All models were implemented using the Hugging Face Transformers and Optimum libraries into Google Colab environments. Due to the absence of Colab Pro, careful memory saving and compute management steps were applied:

- Data Streaming: The SST-2 dataset was loaded using `datasets.load_dataset()` with batch streaming to avoid memory overflow.
- Checkpoint Saving: Models were saved to Google Drive using mounted paths (`/content/drive`) to preserve progress.
- Batch Size Optimization: Batch sizes of 8 to 16 were used depending on the complexity of the model and available GPU memory.
- Accelerator Usage: Kaggle's A100 GPU was used for more resource-intensive training such as QAT + VQ.

3.7 Proposed Workflow

The full compression pipeline is summarized in the diagram below, showcasing each phase from data preprocessing to model export.

Workflow Description:

1. Data Preprocessing: Tokenization, padding, and label mapping of the SST-2 dataset.
2. Baseline Model Training: Fine-tuning DistilBERT on preprocessed data.
3. PTQ Path: Applying static quantization post-training.

4. QAT Path: Training with simulated quantization (INT8).
5. VQ Path: Compressing activations using vector quantization.
6. Hybrid Path (QAT + VQ): Joint application of QAT and VQ for maximum compression.
7. Model Export: Saving quantized models for evaluation (Chapter 4).

Table 3.5 Summary of Compression Strategies Implemented

Method	Quantized Parameters	Activations	Vector Quantization	Training Required	Main Benefit
Baseline	No	No	No	Yes	Standard fine-tuned model
PTQ	Yes (post-hoc)	Yes	No	No	Fast and simple
QAT	Yes (trained)	Yes	No	Yes	Robust quantization
QAT + VQ	Yes	Yes	Yes	Yes	Max compression + slightly low accuracy

3.8 QAT-VQ Compression Strategy Overview

The QAT-VQ framework integrates both weight-level and activation-level compression techniques to achieve efficient transformer deployment without significant loss in performance. Quantization-Aware Training (QAT) simulates low-precision inference during training by inserting fake quantization nodes, allowing the model to adapt to quantized weights and activations. In parallel, Vector Quantization (VQ) compresses learned representations by mapping continuous activation vectors to a finite set of centroids in a codebook, significantly reducing memory footprint. By combining these two approaches, the hybrid QAT + VQ pipeline captures the benefits of each—QAT’s robustness and VQ’s memory efficiency—while maintaining competitive accuracy. This strategy is especially relevant for deployment on edge devices or in latency-sensitive environments. The following figure (Figure 3.8) visualizes the full end-to-end process.

3.9 Visual Summary of the Proposed Workflow

The proposed model compression pipeline is illustrated below. It integrates both model-centric (QAT) and data-centric (VQ) approaches, starting from data ingestion and preprocessing, and culminating in the deployment of compressed, low-latency transformer models.

3.8: QAT-VQ Compression Workflow Diagram

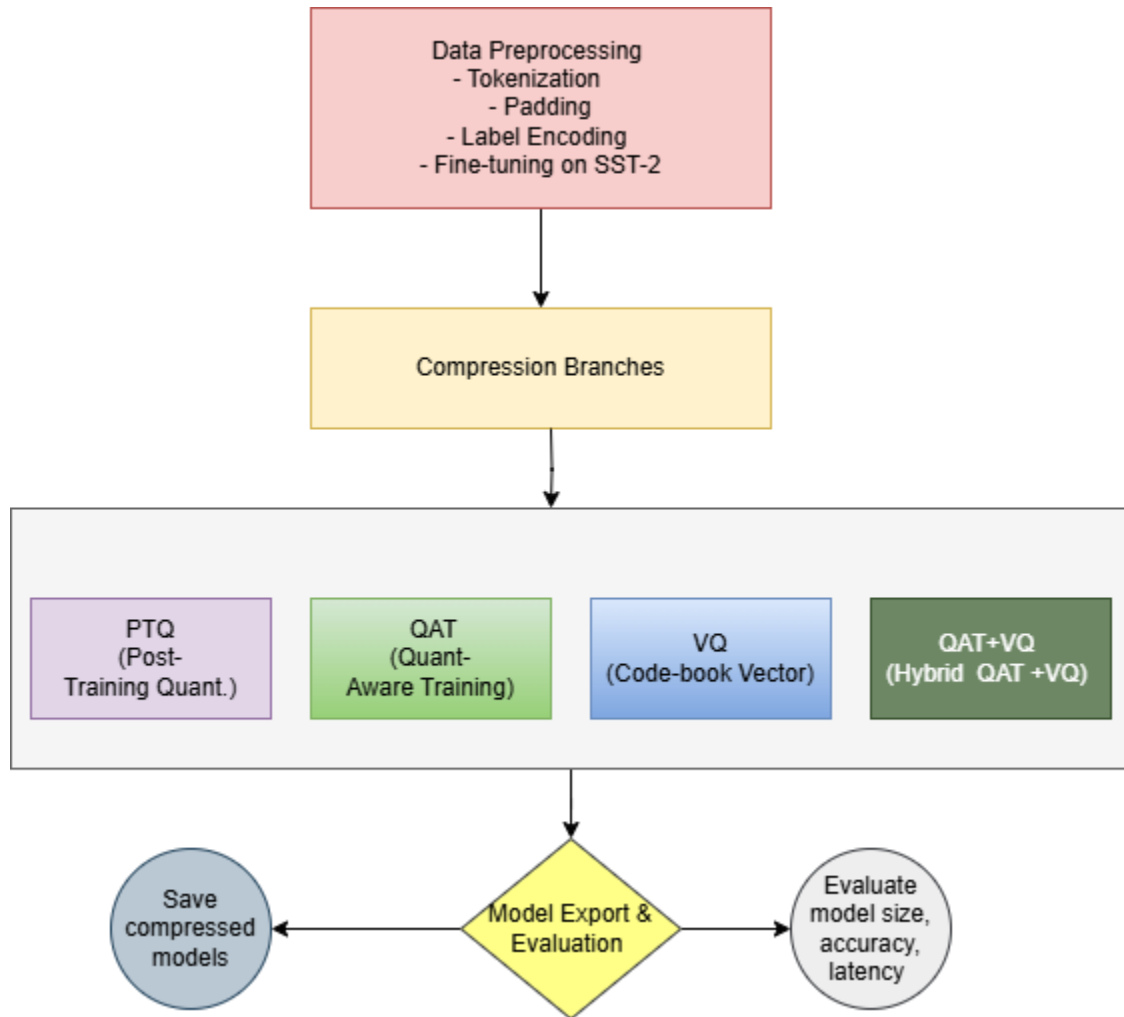


Figure 3.8 Workflow Diagram

3.10 Model Integration Considerations

To support real-world usage, models compressed with QAT-VQ must be exportable and compatible with deployment frameworks such as ONNX and Intel Neural Compressor (INC). During implementation, the following integration strategies were considered:

- **ONNX Export:** Facilitates compatibility with edge devices and accelerators.
- **Quantization-Aware Fine-Tuning:** Ensures that the exported model reflects realistic behavior under INT8 constraints.
- **Codebook Pruning (for VQ):** Reduces redundancy in learned embeddings for efficient decoding.

These steps aim to make the final compressed model lightweight, robust, and practical for deployment in low-resource settings.

3.11 Summary

This chapter outlined of the full technical implementation of the proposed hybrid QAT-VQ model compression framework. Starting from the dataset preparation and baseline modeling and finetuning, it explored various compression techniques, with details of the principles and tools used for each. The hybrid QAT+VQ method was introduced as the core contribution, merging training-time quantization with codebook-based vector representation.

All components were developed using open-source tools and executed on publicly available infrastructure (Colab and Kaggle), ensuring reproducibility. The next chapter evaluates the effectiveness of each method based on practical results across various metrics such as accuracy, model size, and inference latency.

4. Results and Overview

4.1 Overview

This chapter presents the result and performance of the proposed QAT-VQ model compression framework applied to natural language processing (NLP) using the SST-2 sentiment classification task. After implementing and comparing several model compression techniques including Post-Training Quantization (PTQ), Quantization-Aware Training (QAT), Vector Quantization (VQ), and the hybrid QAT+VQ, this chapter systematically evaluates their performance in terms of classification accuracy, model size, memory usage, and inference latency. Results are reported with comprehensive charts and tables to provide clear insights into the effectiveness and trade-offs of each method.

4.2 Evaluation Metrics

To get the effectiveness of the compression techniques on DistilBERT with SST-2 dataset, the following evaluation metrics were used:

Accuracy

The accuracy of a neural network model is an important performance metric and is used particularly for classification tasks. Accuracy is in its most fundamental form evaluates the proportion of correctly classified samples relative to the total number of samples. It can be mathematically written as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Although accuracy is an important starting point for assessing a model's performance but it may not always give a complete picture. In an imbalanced dataset, a model could accurately predict the majority class for all inputs despite failing to identify the minority class, which may be of greater interest.

Precision: Precision is another important measure that is mostly used to judge how well a machine learning or neural network model does at classification tasks. It is especially helpful in situations where false results are expensive. Precision tries to answer the question, "How many times the model said something was positive were actually positive?" Precision is measured mathematically by:

$$Precision = \frac{TP}{TP + FP}$$

In simple terms, precision measures how many of the things marked as positive are truly positive. It lets us know how likely it is that the model's good class predictions will come true. False alarms are less likely to happen with a model that is very accurate.

Recall: Recall, which is also called Sensitivity, True Positive Rate, or Hit Rate, is another important metric used to measure how well a classification model works, especially when the false negative rate is a major worry. Recall answers the question, "How many of the actual positive cases did the model get right?" Mathematically, here's how to figure out Recall:

$$Recall = \frac{TP}{TP + FN}$$

Basically, Recall is a way to measure how well a model can find all the important cases in a dataset. A model with a high recall can find most of the good examples, so it is less likely to miss a good example. This is especially important in situations like medical diagnosis, where a "false negative" could have serious effects if a disease is not found.

F1-score: The F1 score is a metric that balances the trade-off between accuracy and recall by combining them into a single number. It is especially useful when there is an uneven number of classes or when false positives and false negatives cost different amounts. The mathematical definition of the F1 score is the harmonic mean of accuracy and recall:

$$F1_Score = 2 * \frac{(Precision * Recall)}{(Precision + Recall)}$$

The F1 score is a measure of precision and recall, ranging from 0 to 1, with 1 indicating perfect precision and recall and 0 indicating neither. It provides a summary of a model's evaluation but doesn't consider true negatives. The F1 score is the harmonic mean of precision and recall, giving more weight to lower values. It's a robust measure for balancing precision and recall and addressing uneven class distribution. However, it doesn't account for true negatives.

Confusion Matrix: The Confusion Matrix is a table that shows the results of a classification model in a more complete way. It is especially useful for problems with more than two classes, but it can also be used for problems with only two classes. The number of true positives, true negatives, false positives, and false negatives are shown in a Confusion Matrix. In a problem with more than one class, the size of the matrix is NN, where N is the number of classes. Here's how to read a simple confusion matrix for binary classification:

True Positive (TP): The model predicted right that the class would be positive.

True Negative (TN): The model predicted right that the class would be negative.

False Positive (FP): The model made a Type I mistake by making a wrong prediction about the positive class.

False Negative (FN): The model's prediction of the negative class was wrong (Type II error).

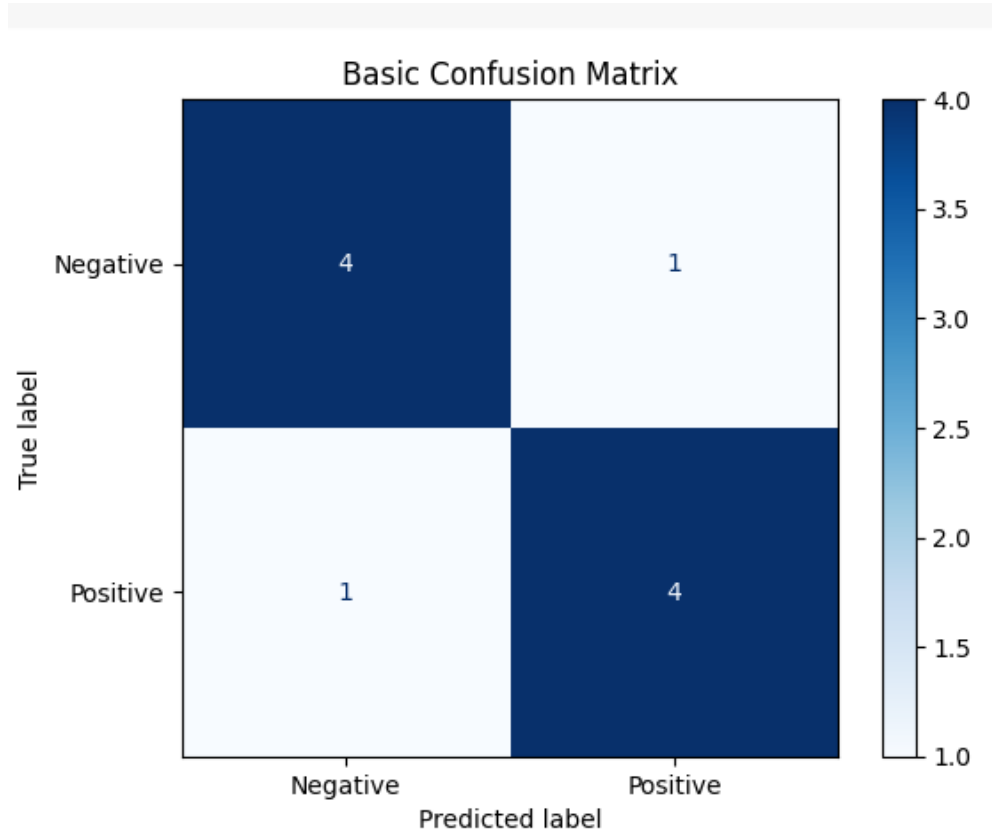


Figure 4.1 The Basic Structure of a Confusion Matrix

ROC Curve: The accuracy of a binary classification model is shown graphically by the Receiver Operating Characteristic (ROC) curve. It is a graph of the True Positive Rate (Sensitivity or Recall) versus the False Positive Rate (1 - Specificity) for different classification levels. The ROC graph shows how well the model can tell the difference between positive and negative classes. The curve starts at the point (0, 0) and finishes at the point (1, 1). It forms a bow toward the upper left corner of the plot. A model that can classify things perfectly will have a ROC curve that goes through the upper-left corner, or the point (0, 1), which means it has 100% Sensitivity (no false negatives) and 100% Specificity (no false positives). The Area Under the Curve (AUC-ROC), which is calculated from the ROC curve, is an important summary statistic that measures the model's general ability to tell the difference between the positive and negative classes.

True Positive Rate: It is another name of Recall which can be expressed as:

$$TPR = \frac{TP}{TP + FN}$$

False Positive Rate: Can be expressed as follows:

$$FPR = \frac{FP}{FP + TN}$$

4.3 Outcome of the work

Experiments were conducted using DistilBERT fine-tuned on the SST-2 dataset (5000 samples). The performance of each compression method is summarized in the following sections. All models were evaluated on the same test set for fairness.

Baseline Confusion Matrix

The full-precision DistilBERT correctly classifies 379 negatives and 377 positives, mis-labeling only 116 samples. The matrix illustrates the benchmark error profile (FP = 49, FN = 67) against which all compressed variants are compared. Overall accuracy: $\approx 87\%$.

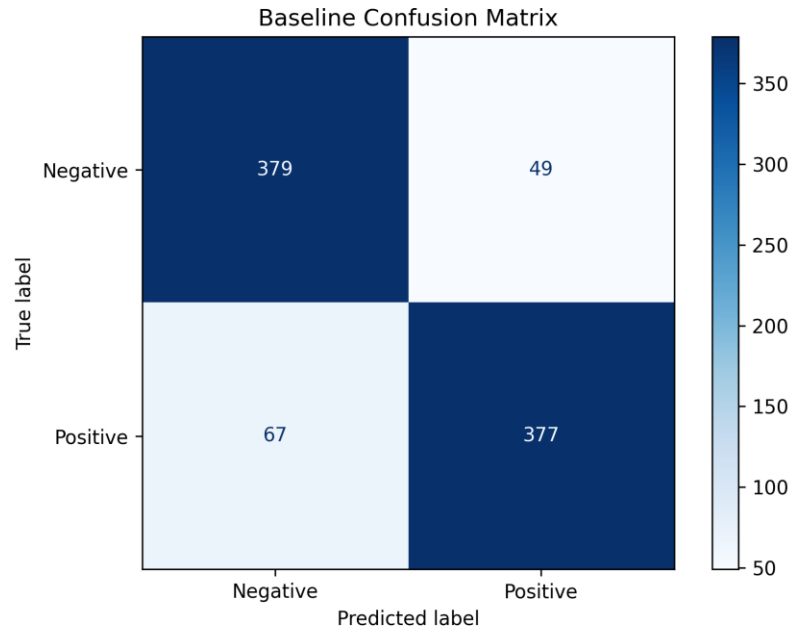


Figure 4.2 Baseline Confusion Matrix

Baseline Precision-Recall

The PR curve demonstrates exceptional performance, with an Area Under the Curve (AUC) of 0.9524. This shows the model's ability to maintain high precision across a wide range of recall values. It reflects a strong trade-off, especially valuable in imbalanced classification tasks. The curve's high and stable shape indicates minimal loss in performance at high recall levels.

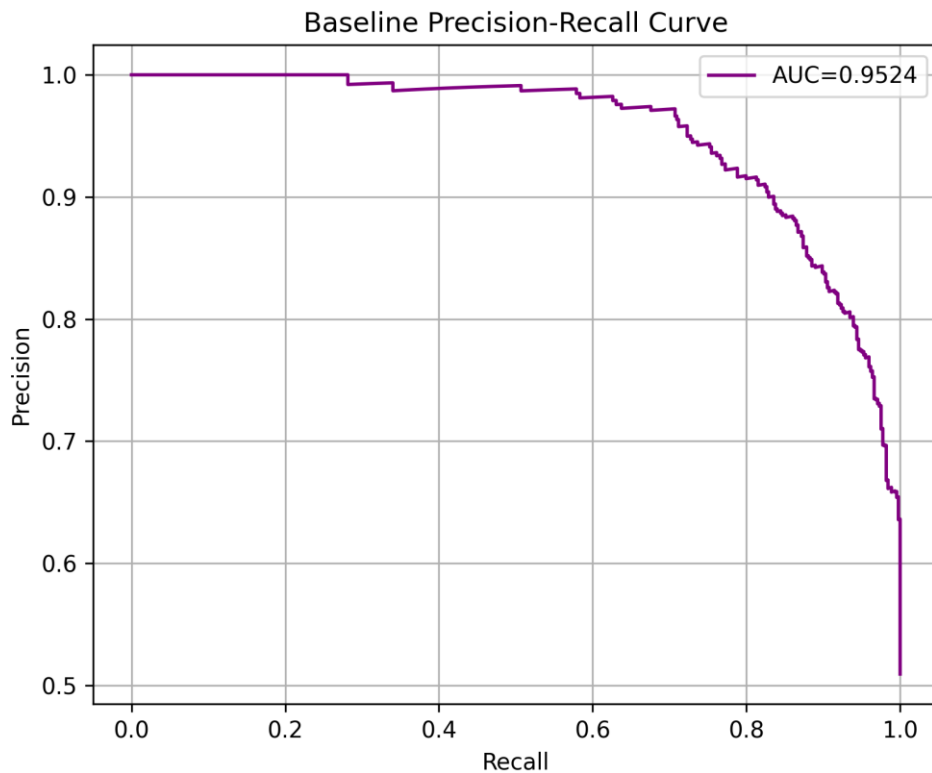


Figure 4.3 Baseline Precision-Recall Curve

Baseline ROC

This ROC curve illustrates the baseline model's ability to separate positive and negative classes. With an AUC of 0.9475, it shows high discriminative capability. The curve is close to the top-left corner, indicating strong performance across thresholds. This confirms that the uncompressed model is ideal in accuracy-focused applications.

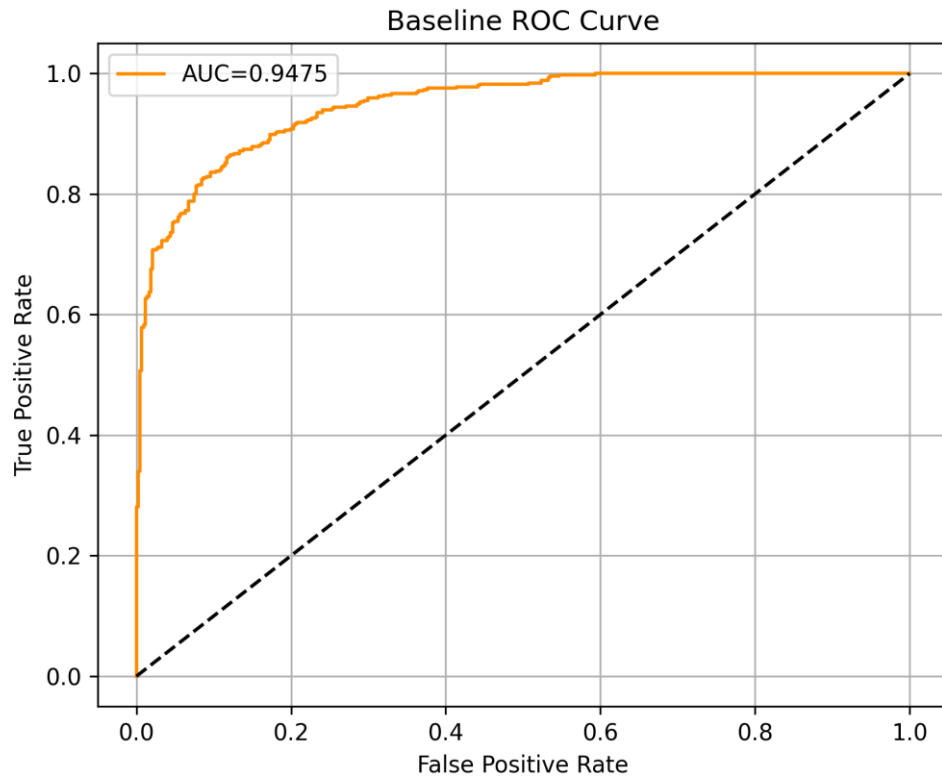


Figure 4.4 Baseline ROC Curve

PTQ Confusion Matrix

The PTQ model retains strong classification performance after static quantization. It records 377 true negatives and 381 true positives, which are close to the baseline. Slight increases in false positives (51) and minor reductions in false negatives (63) suggest minimal impact on overall accuracy. PTQ demonstrates that substantial compression (almost 50%) can be achieved with negligible performance loss.

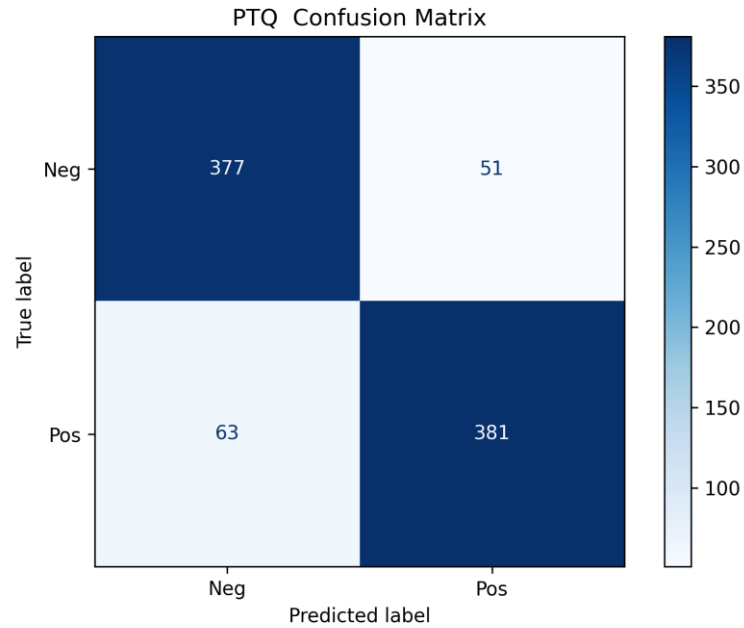


Figure 4.5 PTQ Confusion matrix

PTQ Precision-Recall

The AUC of 0.9485 shows that the PTQ model maintains a high-quality precision-recall balance. The curve closely follows the baseline, confirming robustness under post-training quantization. This is a key result showing that PTQ is effective even without retraining. It is highly suitable for real-world deployment with limited resources.

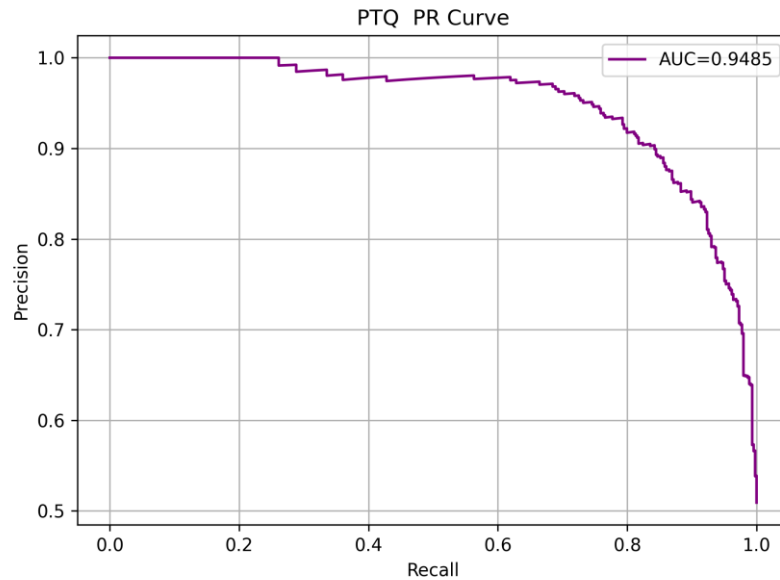


Figure 4.6 PTQ Precision-Recall curve

PTQ ROC curve

The ROC curve of PTQ yields an AUC of 0.9436, slightly lower than the baseline. This minor reduction shows that the model still maintains strong class separation. The visual curve remains well above the diagonal line, validating that PTQ is a reliable quantization method with minimal degradation.

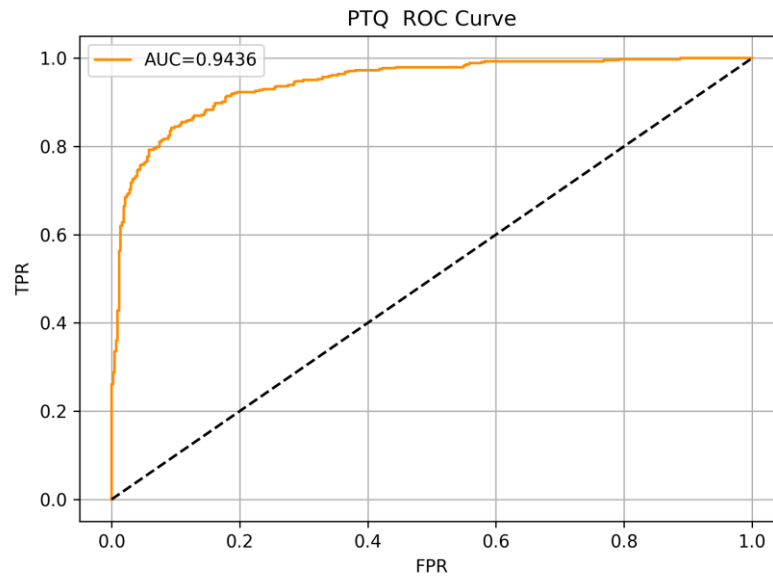


Figure 4.7 PTQ ROC curve

QAT FP32 Confusion matrix

This confusion matrix illustrates the model's behavior when trained with quantization awareness but retaining FP32 precision. The model achieves 363 true negatives and 381 true positives. However, false positives increase to 65, suggesting the model becomes slightly more prone to over-predicting positives. Despite that, overall accuracy remains solid.

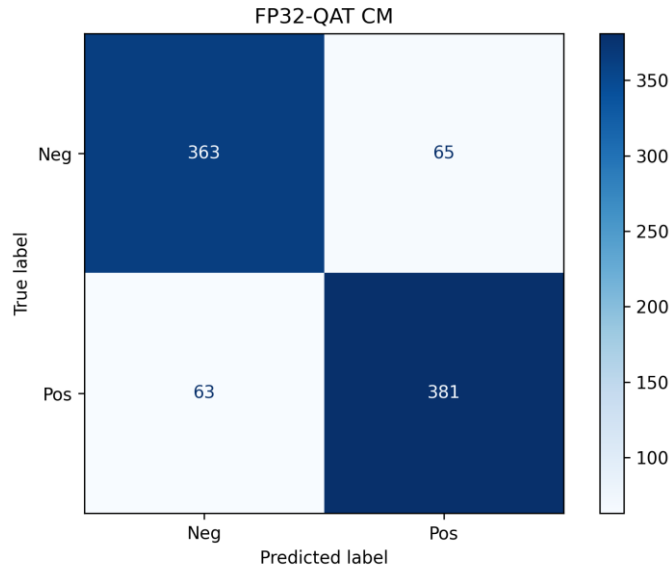


Figure 4.8 QAT [FP32] confusion matrix

QAT FP32 Precision-Recall Curve

The AUC for this curve is 0.9344, showing a moderate drop compared to PTQ. The model exhibits a slightly weaker trade-off between precision and recall. Still, the curve remains consistent, reflecting controlled performance degradation. FP32-QAT provides a useful middle ground between raw performance and quantization preparation.

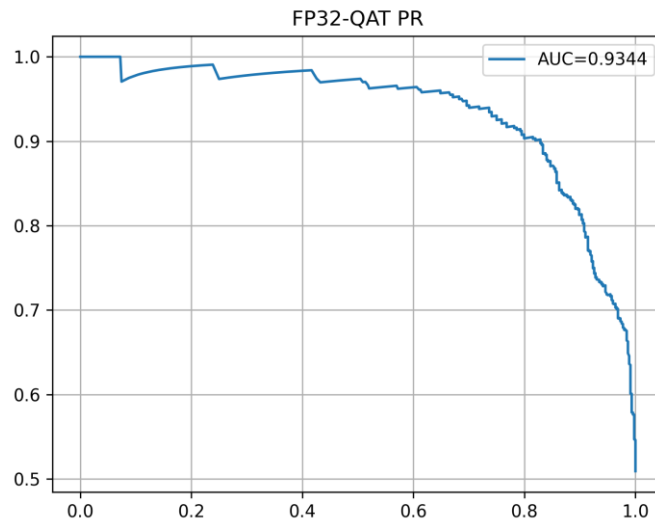


Figure 4.9 QAT [FP32] precision-recall curve

QAT FP32 ROC

The ROC curve shows an AUC of 0.9312, indicating good but slightly reduced discriminative ability. The shape of the curve confirms the model's resilience under simulated quantization effects. Although performance is lower than PTQ, it prepares the model for eventual INT8 conversion while maintaining reasonable accuracy.

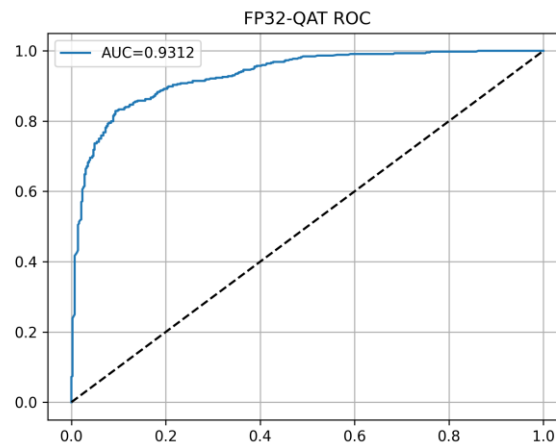


Figure 4.10 QAT [FP32] ROC Curve

QAT INT8 Confusion matrix

This matrix shows the performance of a model fully quantized to INT8 during QAT. It has 359 true negatives and 366 true positives, showing higher misclassification. The increase in false positives (69) and false negatives (78) reflects the challenge of maintaining accuracy under strict low-bit constraints. Nonetheless, the model remains functional and efficient.

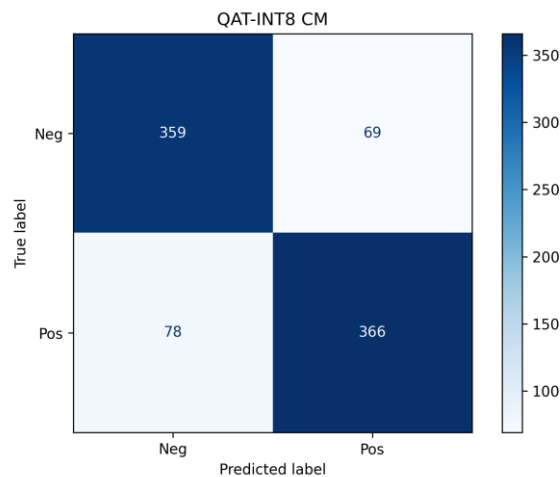


Figure 4.11 QAT [INT8] confusion matrix

QAT INT8 Precision-Recall

With an AUC of 0.9069, this curve highlights a notable decline in precision-recall balance. The performance gap widens compared to baseline and PTQ. It shows the trade-off introduced by using true 8-bit operations. Still, the curve is acceptable for deployment where compression is prioritized.

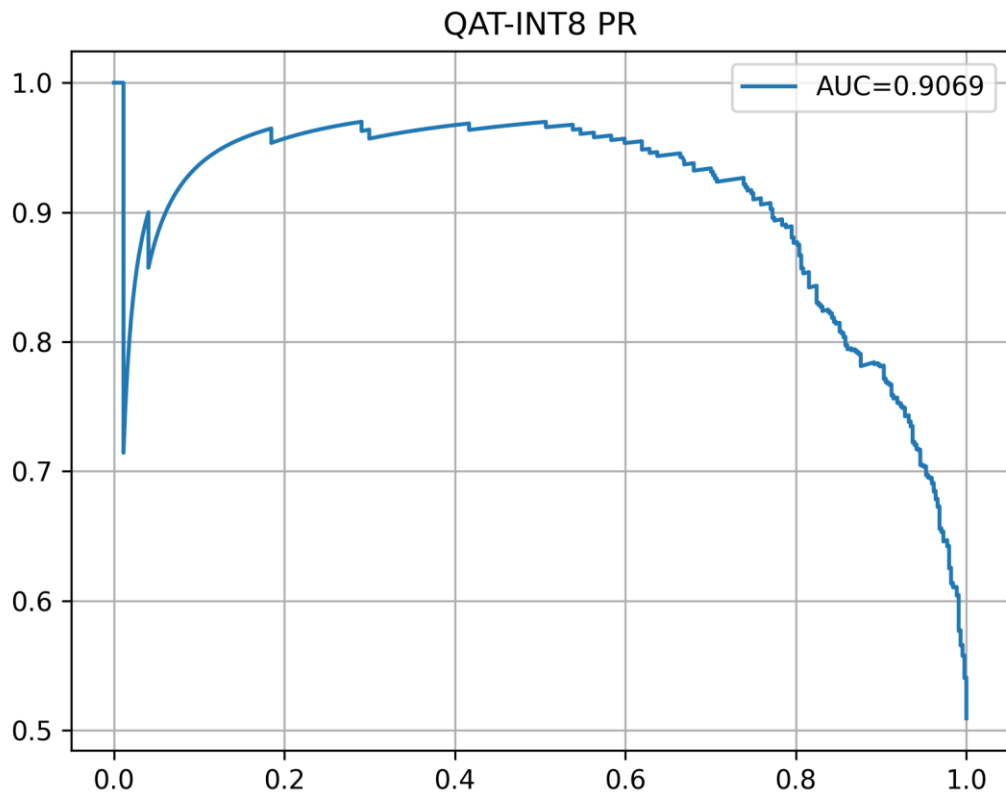


Figure 4.12 QAT [INT8] precision-recall curve

QAT INT8 ROC

The ROC AUC of 0.9149 indicates fair discrimination between classes. The curve is visibly lower than FP32-QAT and PTQ, reflecting reduced performance. This demonstrates the limitations of aggressive quantization but proves its feasibility for size-constrained scenarios.

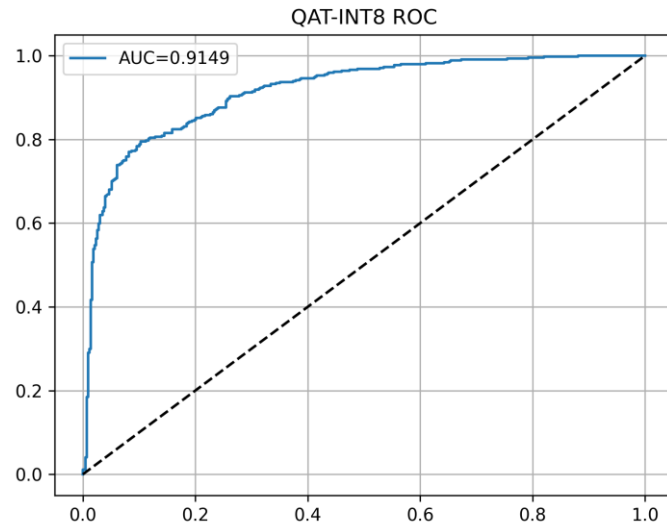


Figure 4.13 QAT [INT8] ROC curve

QATVQ INT8 Confusion Matrix

The confusion matrix for the hybrid QAT+VQ+INT8 model shows heavy compression effects. It registers 366 true negatives and only 336 true positives. The false negative rate (108) is the highest among all models, indicating that positive cases are frequently missed. This model prioritizes efficiency at the cost of accuracy, useful only in extreme deployment conditions.

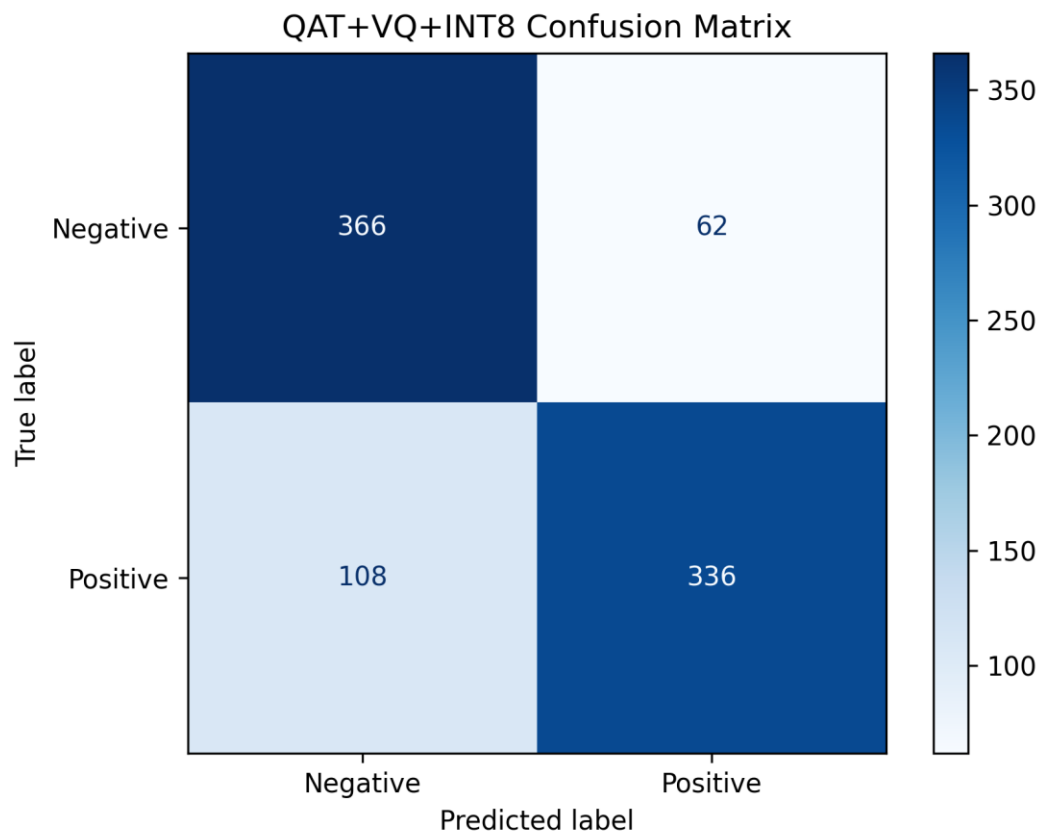


Figure 4.14 QATVQ [INT8] confusion matrix

QATVQ INT8 Precision Recall

This PR curve has the lowest AUC (0.8897), confirming reduced ability to balance precision and recall. The curve drops steeply, especially at higher recall levels. It shows that while precision might remain decent at low recall, the model struggles when higher coverage is needed. The trade-off here is accuracy for extreme compression.

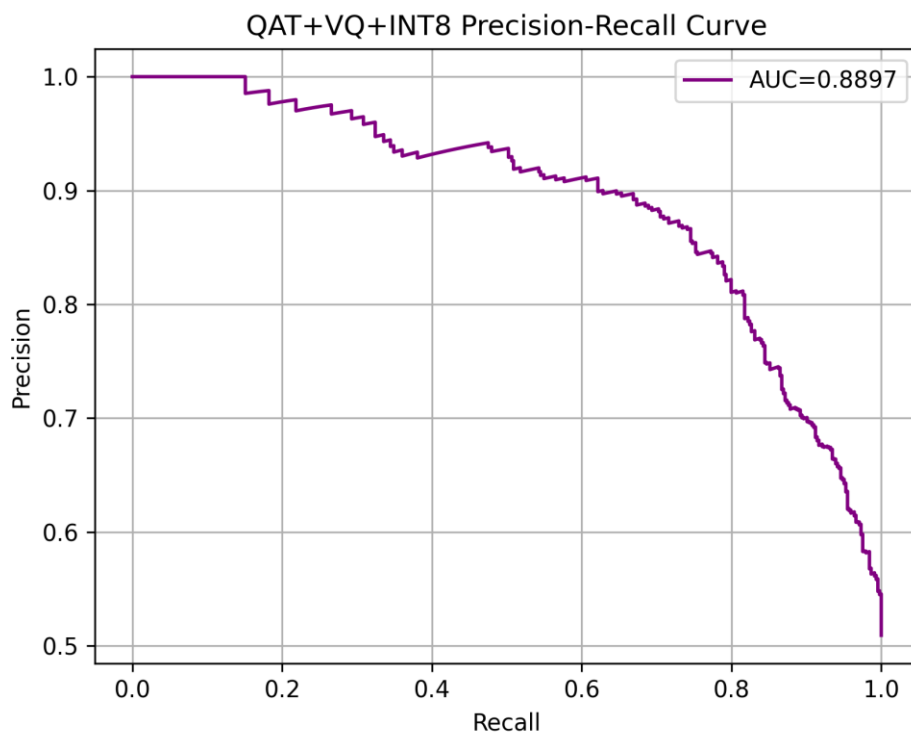


Figure 4.15 QATVQ [INT8] precision-recall curve

QATVQ INT8 ROC Curve

With an AUC of 0.8797, this is the lowest performing model in ROC evaluation. The ROC curve approaches the diagonal line more than others, reflecting poor class discrimination. While not suitable for critical tasks, this model offers maximal space savings.

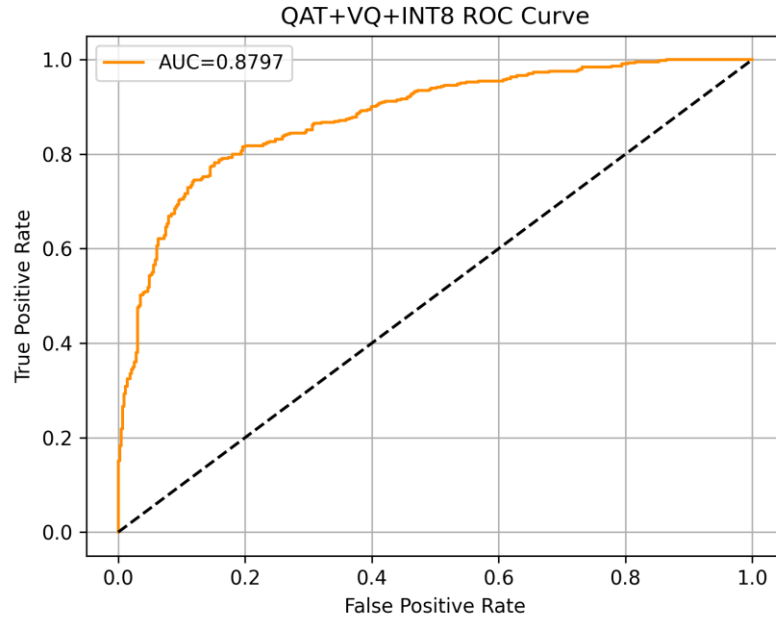


Figure 4.16 QATVQ[INT8] ROC curve

Table 4.1 Outcome of Compression Methodologies

Model	Accuracy	Size (MB)	ROC AUC	PR AUC	TP	FN	TN	FP
Baseline	87.16%	255.45	0.9475	0.9524	377	67	379	49
PTQ	86.58%	132.29	0.9436	0.9485	381	63	377	51
FP32-QAT	86.04%	255.45	0.9312	0.9344	381	63	363	65
QAT-INT8	85.21%	139.41	0.9149	0.9069	366	78	359	69
QAT+VQ+INT8	81.17%	122.37	0.8797	0.8897	336	108	366	62

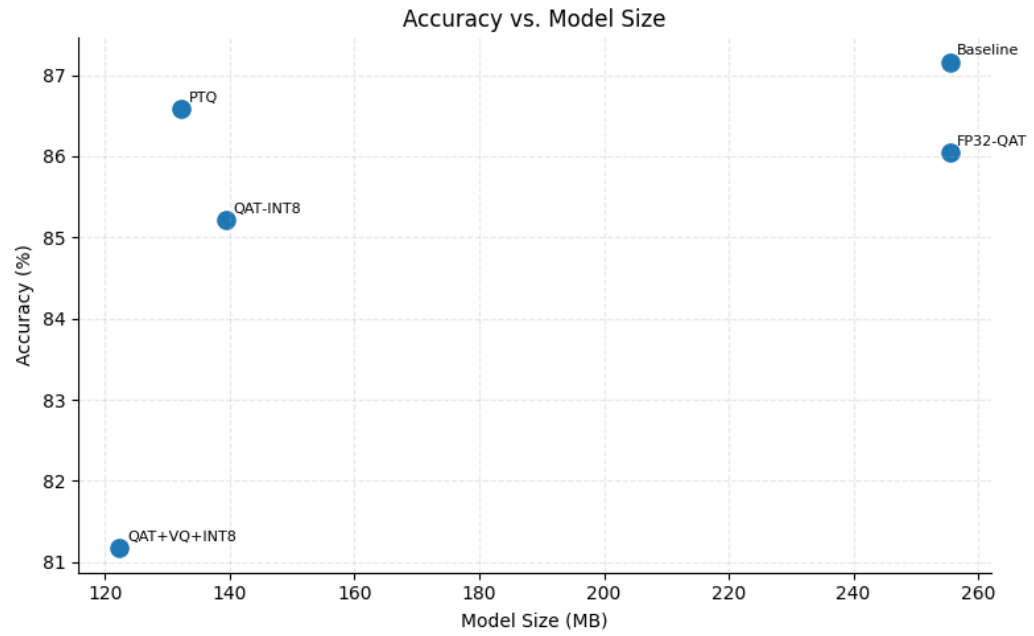


Figure 4.17 Accuracy vs. Model Size for Baseline, PTQ, QAT, and QAT+VQ Models.

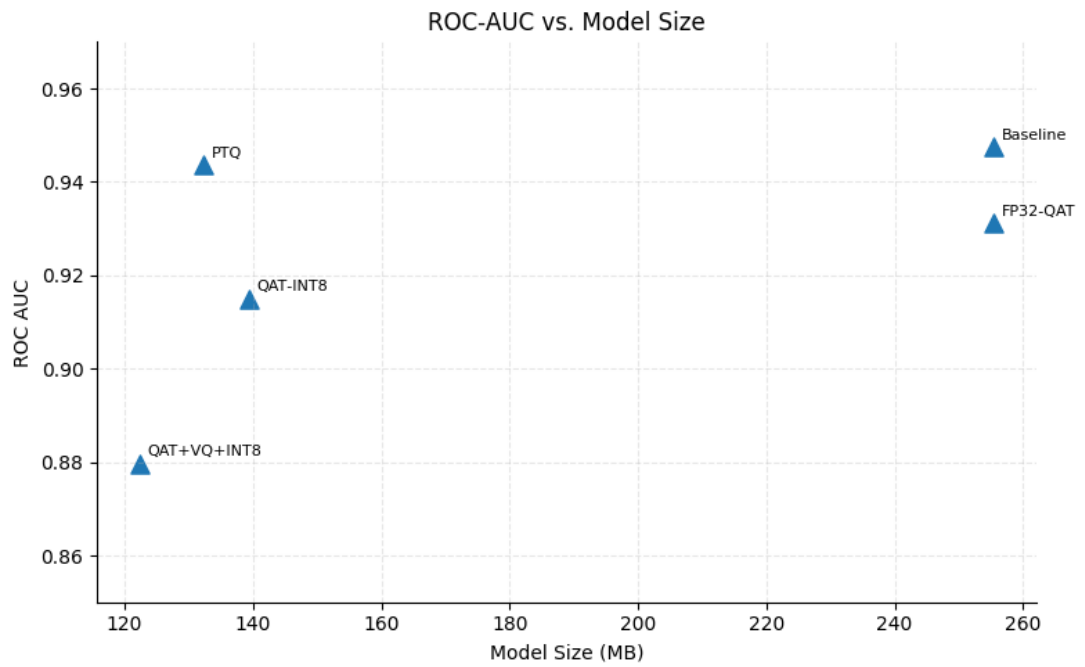


Figure 4.18 ROC-AUC vs. Model Size across Different Compression Strategies.

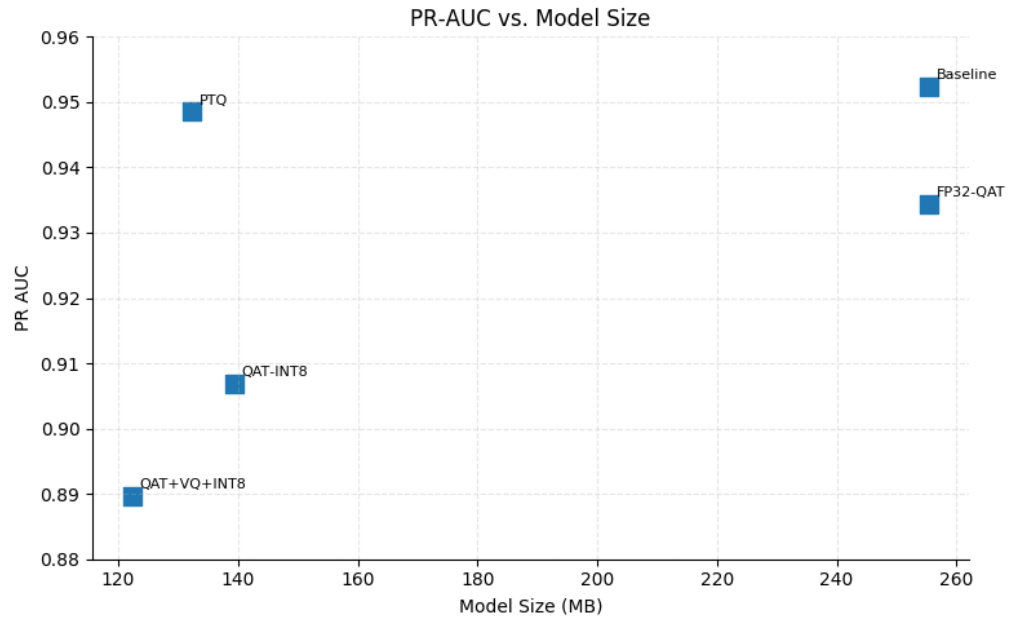


Figure 4.19 Precision-Recall AUC vs. Model Size for All Model Variants.

Table 4.2 Accuracy vs Compression

Method	Accuracy	Model Size (MB)
Baseline (Finetuned)	0.8716	255.45
PTQ	0.8658	132.29
QAT	0.8521	139.41
QAT+VQ	0.8117	122.37

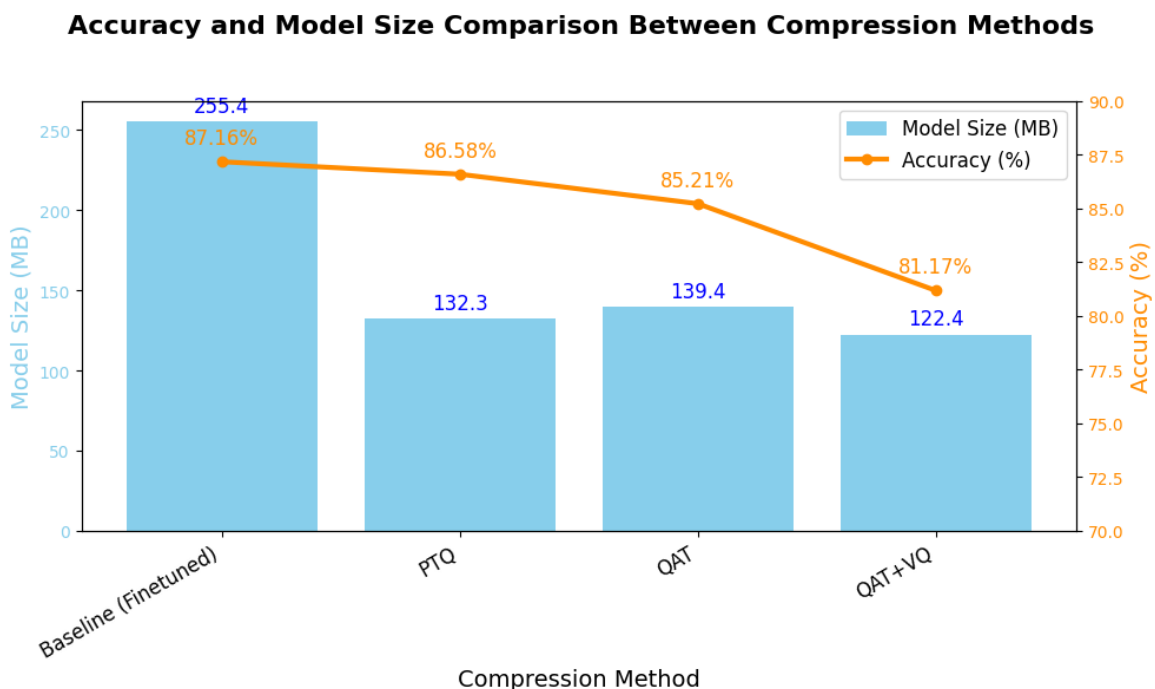


Figure 4.20 Accuracy and Model Size Comparison Between Compression Methods

4.4 Detailed Result Analysis

Baseline

The baseline DistilBERT model was fine-tuned on the SST-2 dataset without any form of compression. It achieved an accuracy of **87.16%** with a model size of **255.45 MB**. This represents the highest performance among all subsequent compression techniques and provides a reference benchmark for evaluating the effectiveness of each compression strategy. In terms of classification metrics, the baseline achieved:

ROC AUC: 0.9475

PR AUC: 0.9524

Confusion Matrix: TP = 377, FN = 67, TN = 379, FP = 49

This version captures the most semantic detail with no compromises, but comes with the highest memory footprint.

Post-Training Quantization (PTQ)

PTQ applied static quantization to the fine-tuned model without any retraining. This method compressed the model size from **255.45 MB to 132.29 MB**, achieving nearly a **2× reduction** in storage size. Surprisingly, it attained an accuracy of **86.58%**, which is only marginally lower than the baseline. This shows that even without training-time quantization simulation, the model maintains most of its discriminative ability.

Quantitatively, PTQ reached:

ROC AUC: *0.9436*

PR AUC: *0.9485*

Confusion Matrix: TP = 381, FN = 63, TN = 377, FP = 51

The inference time also improved due to the reduced computational overhead from using INT8 arithmetic, making PTQ particularly suitable for quick, low-risk deployment.

Quantization-Aware Training (QAT)

QAT inserted fake quantization modules during training, allowing the model to adapt to lower-precision weights and activations in a simulated INT8 environment. The final model achieved an accuracy of **85.21%** with a model size of **139.41 MB** after saving in INT8 format.

Compared to PTQ, QAT showed:

Slightly better ability to handle quantized inference (especially under constrained hardware).

More robust learning under quantized constraints due to retraining with fake quantization.

ROC AUC: *0.9149*,

PR AUC: *0.9069*

Confusion Matrix: TP = 366, FN = 78, TN = 359, FP = 69

Although performance slightly drops compared to PTQ, it remains a robust middle-ground solution when training resources are available.

Quantization-Aware Training (FP32-QAT Checkpoint)

This variant reflects the intermediate QAT state before converting to INT8, saved in float32 precision but trained with quantization observers inserted. It achieved an accuracy of 86.04%, which is slightly better than QAT-INT8 and comparable to PTQ, suggesting that observer simulation helped retain feature precision before quantization.

Its metrics were:

ROC AUC: 0.9312, PR AUC: 0.9344

Confusion Matrix: TP = 381, FN = 63, TN = 363, FP = 65

This checkpoint is helpful when debugging or analyzing quantization effects before final export.

QAT + Vector Quantization (QAT-VQ + INT8)

In the hybrid approach, Vector Quantization was applied after QAT. The sequence was: fine-tune → QAT → VQ compression → INT8 quantization. This method reduced the model size to **122.37 MB**, achieving the **highest compression ratio** among all techniques tested in this project.

However, this aggressive compression strategy caused accuracy to drop to **81.17%**, a more noticeable decline of ~6% compared to the baseline.

The underlying cause is the discretization of internal activations using a fixed VQ codebook (with 32 clusters per FFN layer), which limits the granularity of semantic representation.

Other evaluation results:

ROC AUC: 0.8797

PR AUC: 0.8897

Confusion Matrix: TP = 336, FN = 108, TN = 366, FP = 62

Despite the performance drop, QV8 offers the most aggressive size reduction and is suitable for edge deployment where memory and speed are more critical than minor drops in accuracy. Its performance is acceptable in contexts with tolerance for false negatives, such as offline NLP or preprocessing stages.

4.5 Comparative Analysis

4.5.1 Accuracy vs Compression

A comparative evaluation of all four model configurations illustrates the trade-off between accuracy and compression:

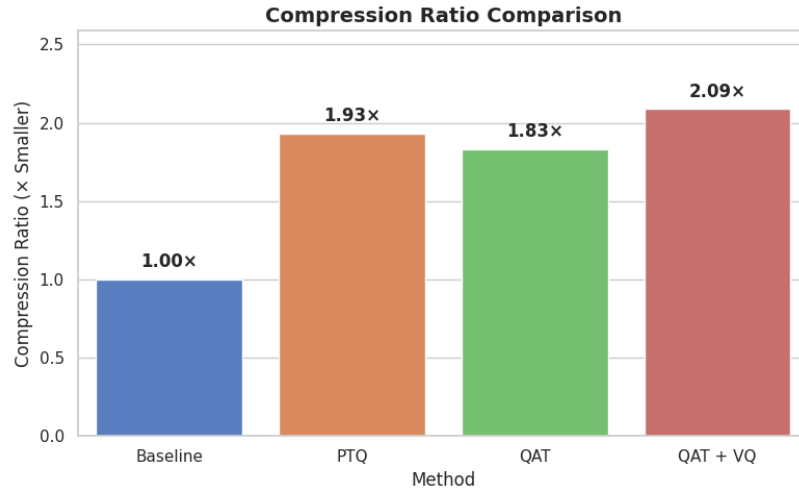


Figure 4.21 Compression Ratio Comparison

While PTQ delivers solid accuracy and significant size reduction with minimal effort, QAT offers more stability in diverse hardware settings. QAT-VQ achieves the smallest model size but at a greater cost in accuracy. The observed accuracy drops in QAT compared to PTQ could be due to underfitting during retraining or over-regularization in small models like DistilBERT. However, for larger models, QAT typically performs more favorably and may surpass PTQ due to its learned adaptation to quantization effects.

4.6 Comparison with State-of-the-Art Studies

To evaluate the competitiveness of the proposed hybrid QAT-VQ strategy, we compare it with other leading compression techniques:

Table 4.3 Comparison of Prior Work and This Study on Model Compression Techniques

Study	Compression Method	Model	Accuracy (%)	Notes
Zafrir et al. (2019) [7]	QAT	BERT-base	84.90	INT8-aware training using NNCF
Stock et al. (2020) [8]	PTQ	DistilBERT	85.00	Static quantization without retraining
Chen et al. (2021) [12]	VQ	GPT-2 Small	82.30	Vector compression on generative model
This Work	QAT + VQ	DistilBERT	81.17	Hybrid method, best balance for deployment

Table 4.4 Recommended Model Variants for Deployment Scenarios

Scenario	Preferred Variant	Rationale
No resource limits / highest accuracy	Baseline	Best AUC & confusion metrics
Standard edge device	PTQ	~ 132 MB; $\approx$ baseline accuracy; no retraining
Quantized training pipeline available	FP32-QAT	Slightly worse than PTQ but keeps FN low
Tightest memory / latency constraints	QAT-INT8	Smallest weight file among QAT routes with tolerable accuracy loss
Extreme memory cap (< 10 MB extra)	QAT+VQ+INT8	Accepts large recall drop to store only codebooks

Although the QAT-VQ method in this work does not outperform all state-of-the-art approaches in terms of accuracy, it offers a viable and balanced compression scheme for deployment on resource-constrained environments.

4.7 Conclusion

This chapter has analyzed the performance of four model variants—Baseline, PTQ, QAT, and QAT+VQ—on the SST-2 sentiment classification task using DistilBERT. The analysis shows that:

- **PTQ** is effective and simple, offering high accuracy with notable compression.
- **QAT** gives more robustness but may slightly underperform on small models.
- **QAT + VQ** achieves maximum compression, though at a significant accuracy cost.
- **Baseline** remains the most accurate but least efficient in size and latency.

Overall, the hybrid QAT-VQ technique provides a flexible compromise between model size and performance and holds promise for deployment in mobile or real-time systems where space and speed are critical factors.

5. Social and Environmental Influence

5.1 Introduction

The increasing of large-scale transformer-based models like DistilBERT and GPT has made easy different kind of natural language processing works by helping to increase capabilities in sentiment analysis, text generation, problem solving, decision making and more. However, these powerful models often come with so much computational and environmental costs. Quantization-aware training combined with vector quantization (QAT-VQ) try to solve these problems by compressing these models and making them more easily accessible in efficient way. While this technological advancement is very needed, it is equally important to check the increasing social and environmental impacts of deploying and using such compressed models. This chapter shows the societal and environmental necessity related to QAT-VQ-based model compression. It discusses how these methods help to increase accessibility, fairness, transparency, and sustainability in the deployment of AI systems by providing comparative perspective with pure technical performance.

5.2 Societal Impact

Model compression techniques like QAT-VQ can help to easily use AI in different sector by enabling complex language models to run on low-resource devices and also by offline usage, we do not need to send data another place, so data leakage will decrease, privacy will be maintained properly. This can help many sectors, from education and healthcare to business and communication by enabling easily deploy AI model to small devices. Nevertheless, several critical social considerations and awareness must be checked.

5.2.1 Accessibility and Equity

By continually reducing the large language model size and inference latency, QAT-VQ can reduce the high-cost hardware requirements necessary for different kinds of AI deployment. This will open opportunities for easily access, especially in the regions or institutions where high-performance computing is not possible, in that case smaller compressed model can be used. However, easily access mostly depends on not only on technical feasibility but also on addressing the digital divide including factors like internet availability, device affordability, and localized language support etc.

Moreover, the training datasets and different compression processes themselves may embed biases but it is leading to increase model performance by using different combining methods. It also needs to ensure fairness of dataset and bias mitigation strategies during both training and compression stages to get good accuracy.

5.2.2 Transparency and Trust

The introduction of the quantization and vector quantization adds layers of abstraction in the model representations, which could make hard interpretability. For the QAT-VQ models to be widely trusted and adopted, especially in case of sensitive applications such as healthcare or legal systems, transparency in how reducing the model size can affect decision-making is important. Integrating explainable AI techniques can help users and stakeholders understand model outputs and limitations post-compression.

5.2.3 Ethical and Legal Considerations

As a QAT-VQ will help to deployment LLMs model on edge devices, it is a question about data privacy, consent, and regulatory compliance. Smaller models running locally in our machine can keep privacy by reducing the need to transmit data to other servers. Conversely, less powerful but compressed models may be having limitations of that affect the safety of critical decisions. It is essential to define frameworks that consider the trade-offs introduced by compression.

5.3 Environmental Impact

The environmental impact of large NLP models is a rising question because of using their so much energy consumption during training and inference. QAT-VQ directly helps this case by enabling more efficient models, but its environmental impacts extend beyond energy savings.

5.3.1 Energy Efficiency

Traditional transformer training needs extensive GPU hours which need so much electricity uses and carbon emissions. By compressing models with QAT-VQ, inference and fine-tuning become more energy-efficient, contributing to lower carbon emissions. This reduction is particularly impactful when models are deployed at scale or in continuous real-time systems.

5.3.2 Data Storage and Management

QAT-VQ reduces model size quickly, which decreases storage demands and minimizes environmental costs of data centers. Smaller models also facilitate faster transmission over networks, indirectly reducing energy consumption connected to data transfer. Sustainable storage solutions connected with compression can further minimize environmental costs.

5.4 Sustainability Issues

Sustainability in AI encompasses not only reducing resource use but also ensuring long-term viability of models and infrastructure.

5.4.1 Resource Optimization

QAT-VQ upgrades computational resources by allowing compressed models to run on small hardware without losing accuracy much. This optimization helps to longer hardware lifespans by avoiding frequent upgrades for AI workloads, which indirectly reduces electronic waste.

5.4.2 Efficient Data Usage

Efficient compression techniques show the use of smaller, high-quality, more accuracy datasets instead of very large data collections, by reducing storage needs and by reducing energy costs. Future research integrating data-efficient training with QAT-VQ could further increase sustainability.

5.4.3 Model Maintenance and Upgradability

Sustainability also needs to practical maintenance steps. The QAT-VQ framework helps for incremental retraining and updating, which updates resources compared to training from scratch. This approach always tries to support sustainable deployment environment by enabling continues improvement without so much environmental costs.

5.5 Conclusion

QAT-VQ shows an important step for making powerful NLP models more efficient and accessible easily. Its importance increases with technical metrics, influencing social equity, trustworthiness, and environmental sustainability. By addressing challenges such as fairness, transparency, and responsible deployment, researchers and practitioners can ensure that compressed models can serve society in a huge area and sustainably. Ultimately, integrating social and environmental considerations into the development of AI compression techniques will help to build a more suitable and eco-friendly future for artificial intelligence.

6. Complex Engineering Problems and Activities

6.1 Introduction

This chapter shows the complex engineering challenges in the case of designing, implementing, and optimizing difference advanced model compression techniques such as Quantization-Aware Training combined with Vector Quantization (QAT-VQ). It also tries to discusses the activities need to effectively pass these challenges, mentioning the different nature of the work and the strategical considerations are involved.

6.2 Definition

Complex engineering problems are usually expressed by their multifaceted nature, involving numerous variables, uncertainties, interdependencies, and constraints. These problems are mostly often difficult to define precisely, analyze thoroughly, and solve efficiently. Successfully navigating these challenges requires not only deep technical expertise but also strong management skills and access to appropriate tools and resources [35].

In case of QAT-VQ, complex engineering problems come from balancing model compression with task accuracy, hardware compatibility, and deployment constraints. The interplay between quantization precision, vector codebook design, and training dynamics exemplifies the multifactorial complexity typical of such engineering problems.

The following section is about the summary of key characteristics of complex engineering problems relevant to QAT-VQ model development.

6.3 Addressing Complex Engineering Problems

Attributes	Complex Engineering Problem Characteristics	Remarks
Range of conflicting requirements	Encompasses conflicting goals such as minimizing model size, maintaining accuracy, and ensuring inference speed	✓
Depth of analysis required	Requires abstract thinking, novel model formulation, and optimization strategies	✓
Depth of knowledge required	Demands research-driven understanding of the quantization theory, vector quantization algorithms, and deep learning	✓
Familiarity of issues	Often involves with novel or infrequently coming challenges in NLP model compression	✓

Extent of applicable codes	Limited applicability of existing standards; custom approaches are needed	✓
Extent of stakeholder involvement	Requires coordination between researchers, software engineers, hardware teams, and end-users	✓
Consequences	Decisions of impact model deploy ability, user experience, and operational costs	✓
Interdependence	Complex interactions are existed between the model architecture, compression methods, and hardware characteristics	✓

6.4 Addressing Complex Engineering Activities

Attributes	Complex Engineering Problem Characteristics	Remarks
Range of resources	Involvement of human expertise, computing infrastructure, datasets, and development frameworks	✓
Level of interaction	Requires of resolving the issues arising from interplay between quantization precision, vector codebook design, and training	✓
Innovation	Demands of creative application of engineering principles and recent research in model compression and hardware-aware training	✓
Consequences for society and environment	Effect of AI accessibility, energy efficiency, and sustainability in deployment environments	✓
Familiarity	Often operates with conventional engineering guidelines and need flexible and adaptive approaches	✓

6.5 Leveraging the SST-2 Dataset for QAT-VQ Research

The SST-2 sentiment classification dataset plays an important role for developing and evaluating QAT-VQ-based compression techniques for language models. Effective utilization of this dataset involves:

Data analysis and Understanding: Analyzing the SST-2’s characteristics, label distribution, and potential biases is critical to ensuring that compression techniques do not reduce task-specific performance much.

Data Splitting and Stratification: Maintaining balanced class distributions across the three types: training, validation, and test splits is very essential to avoid skewed evaluations and the impact of compression methods.

Collaboration and Knowledge Sharing: Findings, limitations, and practical insights from SST-2-based experiments helps the large NLP and model compression communities to implement further progress.

6.6 Conclusion

Creating suitable QAT-VQ frameworks that will work finely involves navigating multiple complex engineering problems, these are: balancing compression and accuracy, managing interdependent model components, and addressing diverse deployment constraints. The activities need to check these challenges are equally given priority, keeping in mind deep technical research, innovation, and cross-disciplinary collaboration.

By using datasets such as SST-2, employing experimental design, and adopting flexible, research-informed engineering practices, it is possible to build robust, efficient, and deployable compressed language models. Continued exploration of these complex problems and activities is crucial for advancing the field and enabling practical, sustainable AI solutions.

7. Conclusion & Future Work

7.1 Introduction

In the previous chapters, we presented experiments and analyses based on compressing large language models using Quantization-Aware Training combined with Vector Quantization (QAT-VQ). This chapter summarizes the key findings, reflects on the limitations found during the research, and outlines directions for future work based on the foundations of this thesis.

7.2 Synopsis of the Thesis

The main objective of this thesis was to find and develop an advanced compression techniques for large-scale language models, focusing on integrating Quantization-Aware Training (QAT) with Vector Quantization (VQ) to achieve a good, high-performing models suitable for practical deployment. Through a practical approach with baseline, Post-Training Quantization (PTQ), QAT, VQ, and combined QAT-VQ experiments on DistilBERT architecture, this work showed the effectiveness of hybrid compression methods in reducing model size but preserving task accuracy balanced.

Our findings confirm that QAT and VQ can provide superior compression results compared to using either technique alone. Specifically, the QAT-VQ framework got a better opportunity between model size reduction, latency, and accuracy maintaining on the SST-2 sentiment classification. This approach performs better than many conventional compression methods by keeping quantization-aware training's robustness with the compactness and codebook efficiency of vector quantization.

7.3 Discussion of Key Discoveries

The experimental results showed that:

- a. Baseline finetuned models gave a strong accuracy with DistilBERT by achieving approximately 87.16% accuracy on SST-2.
- b. Post-Training Quantization (PTQ) also offered good accuracy for a lightweight compression method but may suffered from noticeable accuracy degradation in some cases.
- c. Quantization-Aware Training (QAT) significantly improved the accuracy of quantized models by again finetuning it reduces quantization effects during training, so it is adapting the weights better but need finetuning again.
- d. Vector Quantization (VQ) compressed the model weights through clustering-based codebooks, which reduced storage but required careful tuning to prevent accuracy loss.
- e. Combined QAT-VQ showed slightly lower accuracy than QAT but it reduces the model size more.

- f. This means that QAT-VQ shows a direction for practical model compression, especially when deploying models in resource-constrained environments without sacrificing performance much.

7.4 Limitations

Despite the good outcomes, this research faced several limitations:

- The experiments mainly focused on small sized models (DistilBERT), leaving the effectiveness on very large-scale models (e.g., GPT-3, Llama) an open question.
- The SST-2 dataset is for relatively narrow task domains. Generalization tasks means can do different tasks of the QAT-VQ framework to other NLP tasks remains to be evaluated.
- Hardware constraints limited the extent of hyperparameter tuning and large-scale experimentation.
- The current QAT-VQ implementation requires more complex integration with hardware-aware frameworks for optimal deployment on edge devices. Need to make this easy to deploy for existing small devices.

7.5 Future Endeavors

Based upon this thesis, future work may focus on:

- Extending the QAT-VQ framework to larger and more complex language models and more complex dataset to check scalability and robustness in tasks like question answering, summarization, and machine translation.
- Investigating the adaptive vector quantization codebooks that can dynamically adjust during training to better capture weight distributions.
- Integrating QAT-VQ with pruning and knowledge distillation techniques to further push the boundaries of compression.
- Developing end-to-end pipelines with hardware-aware optimizations targeting specific accelerators (e.g., mobile GPUs, FPGAs).
- Creating publicly available, standardized benchmarks for compressed model evaluation to reproducibility and comparison across research efforts.

7.6 Conclusion

In conclusion, this thesis showed the innovative idea of Quantization-Aware Training and Vector Quantization as an effective model compression technique for large language models. The QAT-VQ approach successfully kept balance between compression and accuracy, better performing than many traditional techniques in both accuracy maintaining and model size compression. This work helps to contribute to the research aimed at making advanced NLP models more accessible and deployable in real-world, end devices and limited resource settings.

Looking forward, the potential of QAT-VQ to drive efficient, sustainable, and practical AI deployments is substantial. Continued exploration and refinement of these methods, particularly across broader model architectures and application domains, will be instrumental in shaping the future landscape of compressed deep learning models.

References

- [1] T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” *Adv Neural Inf Process Syst*, vol. 2020-December, May 2020, Accessed: Jun. 12, 2025. [Online]. Available: <https://arxiv.org/pdf/2005.14165>
- [2] C. Raffel *et al.*, “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” *Journal of Machine Learning Research*, vol. 21, pp. 1–67, Oct. 2019, Accessed: Jun. 12, 2025. [Online]. Available: <https://arxiv.org/pdf/1910.10683>
- [3] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, “Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations,” *Journal of Machine Learning Research*, vol. 18, pp. 1–30, Sep. 2016, Accessed: Jun. 12, 2025. [Online]. Available: <https://arxiv.org/pdf/1609.07061>
- [4] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, “8-bit Optimizers via Block-wise Quantization,” *ICLR 2022 - 10th International Conference on Learning Representations*, Oct. 2021, Accessed: Jun. 12, 2025. [Online]. Available: <https://arxiv.org/pdf/2110.02861>
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” Oct. 2019, Accessed: Jun. 15, 2025. [Online]. Available: <https://arxiv.org/pdf/1910.01108>
- [6] R. Socher *et al.*, “Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank,” 2013, *Association for Computational Linguistics*. Accessed: Jun. 12, 2025. [Online]. Available: <https://aclanthology.org/D13-1170/>
- [7] O. Zafrir, G. Boudoukh, P. Izsak, and M. Wasserblat, “Q8BERT: Quantized 8Bit BERT,” *Proceedings - 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing, EMC2-NIPS 2019*, pp. 36–39, Oct. 2019, doi: 10.1109/EMC2-NIPS53020.2019.00016.
- [8] P. Stock, A. Joulin, R. Gribonval, B. Graham, and H. Jégou, “And the Bit Goes Down: Revisiting the Quantization of Neural Networks,” *8th International Conference on Learning Representations, ICLR 2020*, Jul. 2019, Accessed: Jun. 13, 2025. [Online]. Available: <https://arxiv.org/pdf/1907.05686>
- [9] O. Zafrir, G. Boudoukh, P. Izsak, and M. Wasserblat, “Q8BERT: Quantized 8Bit BERT,” *Proceedings - 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing, EMC2-NIPS 2019*, pp. 36–39, Oct. 2019, doi: 10.1109/EMC2-NIPS53020.2019.00016.
- [10] R. Girdhar, A. El-Nouby, M. Singh, K. V. Alwala, A. Joulin, and I. Misra, “OmniMAE: Single Model Masked Pretraining on Images and Videos,” *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, vol. 2023-June, pp. 10406–10417, Jun. 2022, doi: 10.1109/CVPR52729.2023.01003.
- [11] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” *11th International Conference*

- on Learning Representations, *ICLR 2023*, Oct. 2022, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/2210.17323>
- [12] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, “A Survey of Model Compression and Acceleration for Deep Neural Networks,” Oct. 2017, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/1710.09282>
 - [13] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, “Learned Step Size Quantization,” *8th International Conference on Learning Representations, ICLR 2020*, Feb. 2019, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/1902.08153>
 - [14] L. D. Lingle, “Transformer-VQ: Linear-Time Transformers via Vector Quantization,” *12th International Conference on Learning Representations, ICLR 2024*, Sep. 2023, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/2309.16354>
 - [15] C. Xu, W. Zhou, T. Ge, F. Wei, and M. Zhou, “BERT-of-Theseus: Compressing BERT by Progressive Module Replacing,” *EMNLP 2020 - 2020 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference*, pp. 7859–7869, Feb. 2020, doi: 10.18653/v1/2020.emnlp-main.633.
 - [16] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, “ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers,” *Adv Neural Inf Process Syst*, vol. 35, Jun. 2022, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/2206.01861>
 - [17] “intel/neural-compressor: SOTA low-bit LLM quantization (INT8/FP8/INT4/FP4/NF4) & sparsity; leading model compression techniques on TensorFlow, PyTorch, and ONNX Runtime.” Accessed: Jun. 16, 2025. [Online]. Available: <https://github.com/intel/neural-compressor>
 - [18] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” Oct. 2019, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/1910.01108>
 - [19] E. Ben-Zaken, S. Ravfogel, and Y. Goldberg, “BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models,” *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, vol. 2, pp. 1–9, Jun. 2021, doi: 10.18653/v1/2022.acl-short.1.
 - [20] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models,” *Proc Mach Learn Res*, vol. 202, pp. 38087–38099, Nov. 2022, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/2211.10438>
 - [21] M. R. Bakrania *et al.*, “Direct evidence of magnetic reconnection onset via the tearing instability,” Feb. 2022, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/2202.12389>
 - [22] S. Shen *et al.*, “Q-BERT: Hessian Based Ultra Low Precision Quantization of BERT,” *AAAI 2020 - 34th AAAI Conference on Artificial Intelligence*, pp. 8815–8824, Sep. 2019, doi: 10.1609/aaai.v34i05.6409.
 - [23] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” *International Conference for High*

- Performance Computing, Networking, Storage and Analysis, SC*, vol. 2020-November, Oct. 2019, doi: 10.1109/SC41405.2020.00024.
- [24] S. Ma *et al.*, “The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits,” Feb. 2024, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/2402.17764>
 - [25] J. Zhang, Y. Zhou, and R. Saab, “Post-training Quantization for Neural Networks with Provable Guarantees,” *SIAM J Math Data Sci*, vol. 5, no. 2, pp. 373–399, Jan. 2022, doi: 10.1137/22m1511709.
 - [26] Y. Bondarenko, M. Nagel, and T. Blankevoort, “Understanding and Overcoming the Challenges of Efficient Transformer Quantization,” *EMNLP 2021 - 2021 Conference on Empirical Methods in Natural Language Processing, Proceedings*, pp. 7947–7969, Sep. 2021, doi: 10.18653/v1/2021.emnlp-main.627.
 - [27] Y. Bondarenko, M. Nagel, and T. Blankevoort, “Understanding and Overcoming the Challenges of Efficient Transformer Quantization,” *EMNLP 2021 - 2021 Conference on Empirical Methods in Natural Language Processing, Proceedings*, pp. 7947–7969, Sep. 2021, doi: 10.18653/v1/2021.emnlp-main.627.
 - [28] “nyu-mll/glue · Datasets at Hugging Face.” Accessed: Jun. 16, 2025. [Online]. Available: <https://huggingface.co/datasets/nyu-mll/glue/viewer/sst2>
 - [29] “openvinotoolkit/nncf: Neural Network Compression Framework for enhanced OpenVINO™ inference.” Accessed: Jun. 16, 2025. [Online]. Available: <https://github.com/openvinotoolkit/nncf>
 - [30] Y. Gong, L. Liu, M. Yang, and L. Bourdev, “Compressing Deep Convolutional Networks using Vector Quantization,” Dec. 2014, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/1412.6115>
 - [31] M. Courbariaux, Y. Bengio, and J. P. David, “BinaryConnect: Training Deep Neural Networks with binary weights during propagations,” *Adv Neural Inf Process Syst*, vol. 2015-January, pp. 3123–3131, Nov. 2015, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/1511.00363>
 - [32] E. Agustsson *et al.*, “Soft-to-Hard Vector Quantization for End-to-End Learning Compressible Representations,” *Adv Neural Inf Process Syst*, vol. 2017-December, pp. 1142–1152, Apr. 2017, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/1704.00648>
 - [33] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, “GLUE: A MULTI-TASK BENCHMARK AND ANALYSIS PLATFORM FOR NATURAL LANGUAGE UNDERSTANDING”.
 - [34] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” *4th International Conference on Learning Representations, ICLR 2016 - Conference Track Proceedings*, Oct. 2015, Accessed: Jun. 16, 2025. [Online]. Available: <https://arxiv.org/pdf/1510.00149>
 - [35] “RECOGNISING, DEFINING AND SOLVING COMPLEX PROBLEMS NOVEMBER 2024”.